UNIVERSITY OF GHANA LEGON

ACCRA GHANA

DEPARTMENT OF COMPUTER ENGINEERING

SCHOOL OF ENGINEERING SCIENCES

FINAL YEAR PROJECT REPORT

ON

DETECTION OF LAND ENCROACHMENT WITHIN A TRANSMISSION LINE CORRIDOR USING CONVOLUTIONAL NEURAL NETWORKS

PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE BACHELOR OF SCIENCE DEGREE IN COMPUTER ENGINEERING

NAME OF STUDENT: **DJARBENG RICHARD**

INDEX NUMBER: **10673547**

NAME OF STUDENT: **COLLINS BREFO GYAMERA**

INDEX NUMBER:**10671724**

NAME OF SUPERVISOR: **DR GODFREY A. MILLS**

DATE OF SUBMISSION: 24th SEPTEMBER, 2021

DETECTION OF LAND ENCROACHMENT WITHIN A TRANSMISSION LINE CORRIDOR USING CONVOLUTIONAL NEURAL NETWORKS

By

Djarbeng Richard (10673547) and **Gyamera Collins Brefo (10671724)**

Submitted to the Department of Computer Engineering in

Partial Fulfilment of the Requirements for the Degree of

Bachelor of Science in Computer Engineering

University of Ghana

24th September, 2021

Name of Student: Djarbeng Richard

Signature of Student: _____

Name of Student: Gyamera Collins Brefo

Signature of Student: _____

Name of Supervisor: Dr Godfrey A. Mills

Signature of Supervisor: _____

Name of Head of Department: Dr Godfrey A. Mills

Signature of Head of Department: _____

DECLARATION OF ORIGINALITY

Department of Computer Engineering

We certify that this thesis is our own work except were indicated by referencing. We also certify that we have read and understood the guidelines on plagiarism in the Computer Engineering undergraduate thesis handbook including the University of Ghana's policy on plagiarism. We have acknowledged in the text of this thesis all sources used. We have also fully referenced including page numbers, all the relevant texts, figures, data, and tables quoted from books, journals, articles, Internet websites, and works of other people. We have not used the services of any professional person or agency to produce this document. We have also not presented the work of any student present or past from other academic or research Institutions. We understand that any false claim in respect of this thesis document will result in disciplinary action in accordance with the regulations of the University of Ghana.

Student Name: .....

Index Number: .....

Student Signature: ..... Date.....

Student Name: .....

Index Number: .....

Student Signature: ..... Date.....

Name of Supervisor(s): .....

Supervisor's Signature: ..... Date.....

ABSTRACT

DETECTION OF LAND ENCROACHMENT WITHIN A TRANSMISSION LINE CORRIDOR USING CONVOLUTIONAL NEURAL NETWORKS

Overhead electricity transmission lines have a corridor which is supposed to be free of any encroachment, to make management of the transmission lines very easy and also to protect human life in the event of any line drop or casualty. In the urban areas where the demand for land is quite high, people often encroach on this corridor, making the maintenance work of the transmission utility quite difficult. Transmission utilities are therefore faced with the challenge of monitoring these corridors. Conventional methods for monitoring the transmission line corridors include occasional visits to site, using vehicles to drive through the length of the corridor. Technologies such as drones or helicopters have also been used. These methods when used for periodic monitoring are time consuming and expensive in relation to the coverage area. This project work provides a monitoring system for identification and real-time detection of unauthorized structures such as shops, housing, etc. in a transmission corridor using satellite imagery and machine learning technologies. The detection system is driven by a convolutional neural network architecture. The network was trained using multiple objects along sample transmission lines. 3600 objects comprising of buildings, shops and lands were used to train and test the network. The system was tested using the transmission line corridor of the Ghana Grid Company (GridCo) from their Volta station at Tema to the East Legon Bulk Supply station to the Achimota BSP. The pilot corridor involves 50 transmission towers with variable span length. The model was able to detect at an accuracy of 97.50%. In order to track changes in structural developments on the transmission corridor, the model was integrated with a visualization tracking system. Satellite images from Sentinel-2 satellites using Google Earth Engine were used. Through this solution, the transmission line utility will be able to remotely monitor their transmission lines in real time using a web-based application to avoid any threatening situation before it occurs.

ACKNOWLEDGEMENT

We wish to express our profound gratitude to the Almighty God for seeing us through this project from the conceptualization of the project, through the design process, to the implementation and testing phase too.

We would also like to thank our supervisor Dr Godfrey Mills for whose support this project would not have begun and who has made such a huge impact in our lives since he first taught us in Level 200.

We also wish to thank our parents Mr. and Mrs. Gyamera, Madam Agyen (of blessed memory) and, Mr. Djarbeng for taking care of us throughout our four years of studies. Their support financially, guidance and encouragement are daily appreciated.

We appreciate the efforts of teaching assistants Henry Osei Agyemang, David Boadi, Eric Edem Dzeha, and Bernard Asare for their diverse support and suggestions.

We also appreciate Mr. Mark Baah (Director of Operations, GRIDCo), Martin Kodjo, a representative of the Director of Engineering of GRIDCo, other supporting engineers from GRIDCo, and Dr. Nii Darko, Director of Technical Services, Ghana Energy Commission, for the technical support and contribution to improving the quality of the work. We are also particularly grateful to GRIDCo for providing us the data on their transmission lines within the pilot project area

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY iii
ABSTRACT iv
ACKNOWLEDGEMENT v
TABLE OF CONTENTS vi
LIST OF FIGURES viii
LIST OF TABLES ix
CHAPTER 1 - INTRODUCTION 1
1.1 Introduction 1
1.2 Right of Way Encroachment Detection 2
1.3 Problem Statement 2
1.4 Project Objectives 3
1.5 Relevance of Project 3
1.6 Outline of Thesis 3
CHAPTER 2 - LITERATURE REVIEW 4
2.1 Introduction 4
2.2 Existing Solutions 4
2.2.1 Conventional Approach 4
2.2.2 Modern Approach 5
2.3 Proposed Solution 10
2.4 Scope of Project 10
CHAPTER 3 - ENCROACHMENT DETECTION SYSTEM DESIGN AND DEVELOPMENT 11
3.1 Introduction 11
3.2 System Overview and Operations 11
3.3 System Requirement and Specification 11
3.4 Engineering Design Solution 12
3.5 Data Gathering 14
3.6 Assumptions for System Design 17
3.7 Detection Model System 17

3.7.1 Dataset and Processing..... 17
3.7.2 Convolutional Neural Network.....18
3.7.3 CNN Model Training and Testing..... 22
3.7.4 Model Performance and Evaluation.....26
3.8 Change Detection System Using Google Earth Engine 28
3.8.1 Change detection system specifications.....29
3.8.2 Satellite Image Pre-processing.....29
3.8.3 Change Detection System Overview..... 30
3.9 Application Software Development 31
3.9.1 Database Design.....31
3.9.2 Web Application Design and Development.....32
3.10 Development Tools and Material Requirements 34
CHAPTER 4 - IMPLEMENTATION AND TESTING.....36
4.1 Introduction 36
4.2 System Implementation Process 36
This section entails selecting a pilot area for the testing 36
4.2.1 Encroachment Detection Application System..... 38
4.3 Detection System Testing and Result 52
CHAPTER 5 - CONCLUSION AND RECOMMENDATION.....54
5.1 Introduction 54
5.2 Conclusions 54
5.3 Recommendations 54
5.4 Observations and Challenges 54
References.....55
Appendices.....57
Appendix A1 Convolutional Neural Network model codes 57
Appendix A2 Image Segmentation and Unification codes 60
Appendix A3 Google Earth Engine codes 64

LIST OF FIGURES

Figure 1 View of a transmission corridor 1
Figure 2 View of an Encroached corridor 2
Figure 3 Results of Ankara change detection 8
Figure 4 Results of YOLO detection 9
Figure 5 Operational flow detection system 13
Figure 6 Detection and Tracking System 13
Figure 7 Block diagram of System 14
Figure 8 Sample of images of dataset obtained from Kaggle website 14
Figure 9 Sample of images of dataset obtained from Google Maps 15
Figure 10 Visualization of a KMZ file in Google Earth Pro. 15
Figure 11 View of metadata information for a KMZ file 16
Figure 12 Snapshot of MS Excel tower locations 16
Figure 13 Image processing flow structure 18
Figure 14 Convolutional Neural Network Architecture 19
Figure 15 Flow of the CNN model design 20
Figure 16 21
Figure 17 Kernel Operation 21
Figure 18 Data Accuracy comparison 23
Figure 19 Model summary 24
Figure 20 Adam training process 25
Figure 21 Adamax training process 25
Figure 22 RMSPROP training process 26
Figure 23 Model Performance 26
Figure 24 Adamax Model Accuracy 27
Figure 25 Adamax Model Loss 27
Figure 26 Model Accuracy 27
Figure 27 Adam Model Loss 28
Figure 28 RMSprop Model accuracy 28
Figure 29 RMSProp Model Loss 28
Figure 30 Comparison of older and recent image band values 29
Figure 31 Satellite image pre-processing flow diagram 30
Figure 32 Image differencing chart results 30
Figure 33 Change detection system flow chart 31
Figure 34 ER diagram 32
Figure 35 Flow diagram of detection system 33
Figure 36 Flow diagram of detection system cont'd 33
Figure 37 Use case diagram 34
Figure 38 Selected image for segmentation 36
Figure 39 Segmented Images 37
Figure 40 Segmented images with contours drawn 37

Figure 41 Login interface 38
Figure 42 Dashboard 39
Figure 43 tower4 to tower5 40
Figure 44 Tower 4 to tower5 after detection 40
Figure 45 Notification Status 41
Figure 46 WhatsApp Notification 41
Figure 47 Tower 13 to Tower 14 42
Figure 48 Tower13 to Tower14 after Detection 42
Figure 49 Tower13 to Tower14 Notification status 43
Figure 50 Tower13 to Tower14 WhatsApp notification 44
Figure 51 Towers superimposed on the map 46
Figure 52 towers superimposed on the map in satellite view 46
Figure 53 Map view with tower location selection 47
Figure 54 Map view showing a highlighted area of the map in satellite view. 47
Figure 55 Transmission line buffer on satellite image 48
Figure 56 Change detection system prototype 48
Figure 57 Encroachment detection in corridor 48
Figure 58 Change detection with thresholding 49
Figure 59 Change detection model in operation 49
Figure 60 Clouds in the buffer 51
Figure 61 Detection confirmation in Google Earth Pro 51
Figure 62 Encroached Area confirmation 2021 53
Figure 63 Encroached Area confirmation 2020 53

LIST OF TABLES

Table 1.1 Comparison of physical corridor inspection methods 5
Table 2 Sample tower location Information 17
Table 3 Dataset information 20
Table 4 Hyper Parameters of CNN system 22
Table 5 Three hidden layers with learning rate of 0.1 22
Table 6 Three hidden layers with learning rate of 0.01 23
Table 7 Three hidden layers with learning rate of 0.001 23
Table 8 Model Evaluation 27
Table 9 Tower 4 to Tower 5 Coordinates 39
Table 10 Tower 13 Tower 14 Coordinates 39
Table 11 Image Classification and Accuracy 52
Table 12 Detected areas of encroachment 53

CHAPTER 1 - INTRODUCTION

1.1 Introduction

Electrical power transmission, ranging from 11kW to 330kW, is mostly done through both overhead lines and underground cables. While underground cables are securely buried underground, the overhead lines are visibly in the open. Whether buried or overhead, these transmission lines require a right of way without any encroachment to ensure that the utility can carry out their line maintenance, operational work and other services on the lines including replacing existing lines without any hindrance. In the case of the overhead lines, the margin of the right of way is given extensive, typically 30 metre width, 15 meters each from the centre of the lines. In the rural or semi-urban areas, these line buffers are not usually encroached because of the availability of vast land. In the urban areas however, where there is demand for land space, these open buffers become the centre of attraction. People tend to construct shops, resident structures, car garages, etc. within the right of way buffer. Due to the problem of encroachment on these transmission corridors, the utilities have had challenges in carrying out maintenance activities. This therefore requires the utilities to carry out occasional checks on all their transmission lines to identify any potential threats and obstacles for redress. For example, in Ghana, the transmission authority, Ghana Grid Company Limited (GRIDCo) occasionally demolishes structures along their transmission corridor to enable them carry out line maintenance or construction of new lines along existing lines. If remote early warning mechanism warning could be deployed which could trigger alerts on any potential encroachment of the transmission line corridors, this could help the transmission utility in the expensive approach of line monitoring. Moreover, a mechanism that could also track changes along the corridor will also help the utility to monitor the advancement or changes within the corridor's right of way.

[1]

Figure 1 View of a transmission corridor

1.2 Right of Way Encroachment Detection

Utilities ensure safe transmission of power by utilizing a buffer around high voltage line. They continuously monitor their transmission corridor's right of way through physical inspection and in some cases resort to helicopters, unmanned aerial vehicles or imaging drone services [2]. Automatic monitoring using technological solutions that could provide timely information of activities on the corridor will provide a more efficient means of monitoring a transmission corridor. In an attempt to ensure that its transmission right of way is clear of all obstruction, utilities use a range of detection approaches. They use physical inspection and technology such as aerial drone technology services, unmanned aerial vehicles, helicopters or satellite imagery.

1.3 Problem Statement

The transmission authority in Ghana, Ghana Grid Company Ltd (GRIDCo), has challenges in timely and effective monitoring of the right-of-way corridor of transmission lines mostly in urban and semi-urban areas. As a result of late monitoring of transmission corridors, there is encroachment within the transmission right-of-way, this brings an additional cost in clearing the encroached areas. It also poses a safety risk in the event of an accident to those who settle close to high voltage power lines. There are also adverse health implications of living close to high tension cables. A study from the British Columbia Medical Journal [3] has linked potential adverse health effects to people who live close to high voltage power lines. The study finds an elevated risk of 23 –69% for developing childhood leukaemia from living within 200m of a high voltage transmission line. By measuring magnetic field strength as an exposure measure the risk of leukaemia was doubled in the range of 60m within a high-voltage power line. The right-of-way buffer set by GRIDCo for its transmission corridor is 50m. By bringing in early monitoring leveraging on technology with AI to provide real time monitoring of the transmission corridor's buffer and detection of objects we could provide a solution to these issues.

Figure 2 View of an Encroached corridor

1.4 Project Objectives

To provide solutions to the proposed problem, the following objectives were to be accomplished:

- Develop a web-application for real time monitoring using satellite images and machine learning techniques to detect encroachment in a chosen corridor.
- Provide a system to provide an alert to the utility of any potential obstruction along the corridor for quick intervention.
- To create functionality for tracking an area view changes in the transmission corridor.

1.5 Relevance of Project

The project has enormous benefit to the transmission utilities the costs associated with fuelling vehicles to carry out physical inspection or drones and helicopters. The issue of manpower requirements to carry out routine physical monitoring is reduced and the right-of-way corridor is kept free for.

1.6 Outline of Thesis

This document is organized into five (5) chapters. Chapter 1 is an introduction of the problem to be solved and its scope of application It provides background information on the project, an overview of the objectives, as well as its significance. The encroachment detection system is meant to provide an easier way to monitor a given transmission corridor. Chapter 2 gives a review of existing works by scholars with similarity to this project. This enables us draw on the work of others before us to avoid duplicating work already done. It also allows us to evaluate the shortcomings and see how to implement the recommendations and gaps found in our project. Chapter 3 provides the steps taken in the design and

development of the encroachment detection system; system overview, assumptions made in building the system, requirements and specifications, application development and the tools used in building the system. Chapter 4 shows the outcome of prototype implementation and testing as well as discussion of the results. Chapter 5 presents the conclusions of the work and system implementation. It includes the challenges encountered in the various system design stages in addition to noted observations and recommendations.

CHAPTER 2 - LITERATURE REVIEW

2.1 Introduction

This chapter is a discussion of existing approaches and similar works in the field of study. The problem of detecting unauthorized structures in an area is not a new one. The approaches have been categorized into two; conventional approaches to corridor monitoring and modern approaches of monitoring using technology and artificial intelligence (AI).

2.2 Existing Solutions

We review existing approaches on encroachment detection to understand the techniques used and to avoid repeating work that has already been done. Some of the works reviewed show different methods for detecting buildings in images, others touched on the most efficient tools to obtain an accurate detection model and some discuss the advantages of satellite monitoring over conventional practices such as using vehicles. Modern approaches use imagery from aircraft, camera drones or Unmanned Aerial Vehicles (UAV), LIDAR data and satellite imagery.

2.2.1 Conventional Approach

A Review of Vegetation Encroachment Detection in Power Transmission Lines using Optical Sensing Satellite Imagery[4]

In this work, Haroun et al compares the varied ways of monitoring the right-of-way corridor for transmission lines.

The work discusses the variety of methods that are in use for transmission corridor monitoring and gives the advantages and disadvantages of each. The authors also discuss ways of using satellite imagery to monitor the right-of-way corridor. They also observe that although using imaging drones for monitoring is becoming more common, they are limited in range due to battery capacity and are unsuitable for long distance applications. They also discuss the application of machine learning in addition to deep learning to create a dynamic detection process. This is because current methods for transmission line monitoring from images rely on manually setting threshold values instead of allowing the system to learn and generalize the features in the dataset. The authors suggest “Machine Learning (ML) and Deep Learning (DL) algorithms can provide a very high accuracy with flexibility in the detection process.”
The findings for power line inspection techniques and their downsides are summarized in the table below.

Table 1.1 Comparison of physical corridor inspection methods

TECHNIQUE	PROBLEM

LIGHT DETECTION AND RANGING	SLOW, LABOURIOUS IN DIFFICULT TERRAIN
SYNTHETIC APERTURE RADAR	EXPENSIVE IN RELATION TO COVERAGE AREA
UNMANNED AERIAL	EXPENSIVE IN RELATION TO COVERAGE AREA
VEHICLES(UAV) AND AIRBONE FACILITIES	EXPENSIVE IN RELATION TO COVERAGE AREA

The authors conclude thus, “using satellite images can replace the other relatively expensive methods like LiDAR and airborne monitoring”

2.2.2 Modern Approach

We consider works that used technology or AI based approaches for monitoring. Ostankovich and Afanasyev [5] looked at a process of automating the inspection of buildings using a state cadastral map and Google Net deep learning tools by combining four building detection methods. Shunping et al [6]proposed a new building change detection framework which produces binary outputs as inputs to a self-trained building change detection network for Very High Resolution (VHR) images. Celik [7] used a combination of Synthetic Aperture Radar (SAR) and Multi-Spectral Instrument (MSI) data from Sentinel-1 and Sentinel-2 satellites to analyze change in urbanization in Turkey’s capital using Google Earth Engine (GEE). Lefebvre et al.[8] conducted image classification and image fusion between Sentinel and Landsat images based on evidence theory (Dempster–Shafer) for change detection in the cities of Prague and Rennes. The findings of these works and others are discussed in more detail in the subsequent subsections.

Illegal Buildings Detection from Satellite Images using Google Net and Cadastral Map[5]

Ostankovich and Afanasyev propose a methodology that incorporates image processing techniques with deep learning tools for the automation of state inspection of illegal buildings.

Methodology:

In this work, image pre-processing is mainly described by vegetation removal. Since vegetation assumes green dominant colour, the red and blue bands of RGB image were assumed to have lower values than the green band. They apply an inverse mask to obtain a new image, which is assumed to have no vegetation. Then the processed image is used in four existing building detection approaches;

- **Shadow detection**
- **Canny Edge Detector**
- **CIELUV Colour space**

- **Active Contours Model**

These approaches produce binary masks as outputs from which duplicates are removed. Bounding boxes are obtained after the binary classification step and classified as buildings. A trained Google Net Convolutional Neural Network (CNN) classifies the regions into building or background classes. Building footprints were detected in satellite images and compared with a state cadastral map. The building objects are assessed for legality via the open Russian Federation State Cadastral Map (It is a web-based map, which can give information for a GPS location.)

Results:

The system reached a building detection accuracy of 89.79% and precision of 92.83%. The average percentage of false positives (FP) considered legal was found to be around 8.8% based on all test images

.

Figure 2.1 Results of building detection

Building Instance Change Detection from Large-Scale Aerial Images using Convolutional Neural Networks and Simulated Samples[6]

Shunping et al. focus on a new building change detection framework with a building extraction network and a self-trained building change detection network for Very High-Resolution remote sensing images.

Methodology:

Shunping et al. divide the change detection process into three major steps: pre-processing, change detection technique selection and accuracy assessment. The pre-processing stage primarily deals with

issues related to atmospheric and radiometric correction, topographic correction and image registration. A Convolutional Neural Network (CNN), a branch of artificial learning, was also added to the system for change detection. The system proved to have a higher efficiency than its predecessors demonstrating the CNN as the preferred method for image classification. The building change detection network takes bi-temporal building classification maps as inputs and computes building change maps at the pixel and object levels. The input bi-temporal images are classified into a binary map of buildings and background using a building extraction network. The noises in the maps caused by imperfect building extraction were filtered out with a designed filter, then the maps of different times were concatenated and input into the change detection network to detect building changes and produce a building change map. Binary maps produced from the building extraction network were used as input for the Change Detection Network. First, the binary maps were simulated to detect the building changes. Secondly, the registration of building errors in the binary maps by randomly shifting a building's mask within a given threshold was used to train the network that is resistant to this change is simulated.

Results:

The binary images having gone through the process and the CNN model being applied to it the system reached an overall precision of 0.856 and a total recall of 92.2%

Change Detection of Urban Areas in Ankara through Google Earth Engine [7]

Celik utilized Sentinel-1 and Sentinel-2 images to analyse the change in urbanization in 21 rapidly developing neighbourhood units in the capital city of Turkey, Ankara, from 2015 to 2017.

Methodology:

The author performed image subtraction on two images from 2015 and 2017. The images were obtained by combining SAR and MSI imagery from two satellite sources. Indices such as Difference Built-up Index (NDBI), Bare Soil Index (BSI) and Soil-adjusted Vegetation Index (SAVI) were also used. Binary supervised classification was performed by using a random forest classifier before post-processing with morphological operators to achieve greater accuracy.

Results:

The study achieved a 91% overall test accuracy upon post-processing and a kappa value of 0.82 showing an average of 5.9% change of the total area. By utilizing a combination of "fill minima", "focal_min", "focal_max" and filtered clusters to remove changes less than 5 pixels the system obtained an overall accuracy ranging from 72% to 92%.

One setback the author faced was the problem of differentiating agricultural change from urban change in the two images. Another was the inability to recognize small size buildings using a number of iterations of "focal_min" operation.

The results of application tests are shown below:

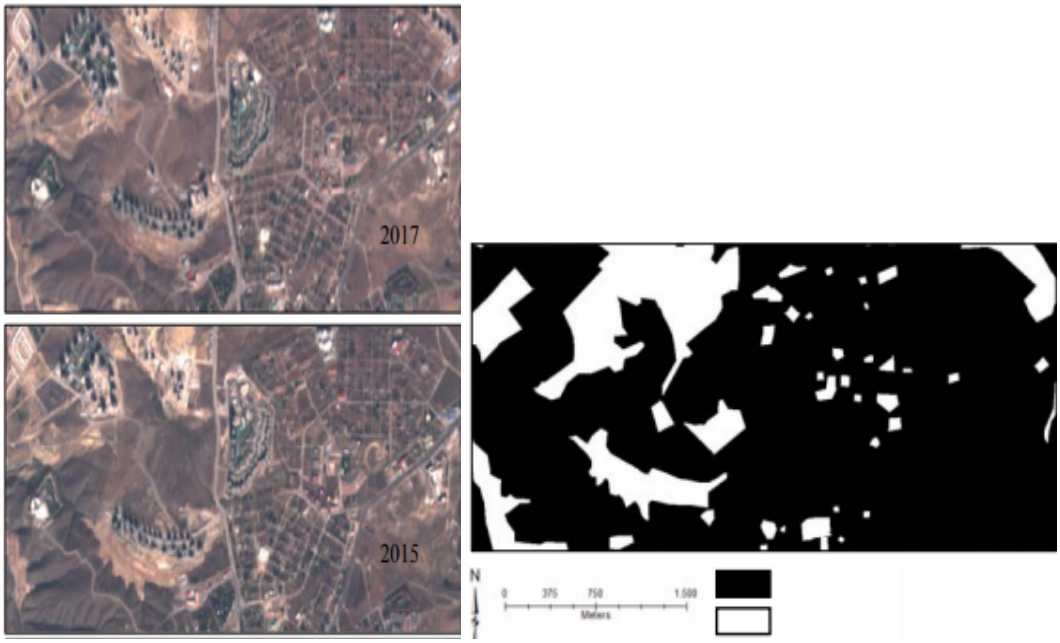


Figure 3 Results of Ankara change detection

You Only Look Once: Unified, Real-Time Object Detection[9]

The paper introduces a new approach to object detection where a model can look at an entire image in a single pass to predict what objects are present and where they are.

Methodology:

Redmon et al. used a single neural network to predict bounding boxes and class probabilities directly from full images in one evaluation. YOLO sees the entire image during training and test time so it implicitly encodes contextual information about classes as well as their appearance. A model was implemented using convolutional neural networks (24 convolutional layers followed by 2 fully connected layers) and evaluated on the PASCAL VOC detection dataset. The point of the work was to treat object detection as a regression problem to spatially separated bounding boxes and associated class probabilities.

Results:

While it can quickly identify objects in images it struggles to precisely localize small objects especially those in groups however it is less likely to detect false positives outperforms other detection methods, including DPM and R-CNN. By adding YOLO to an R-CNN the authors achieved an accuracy increase of 3.2% to 75.0%.

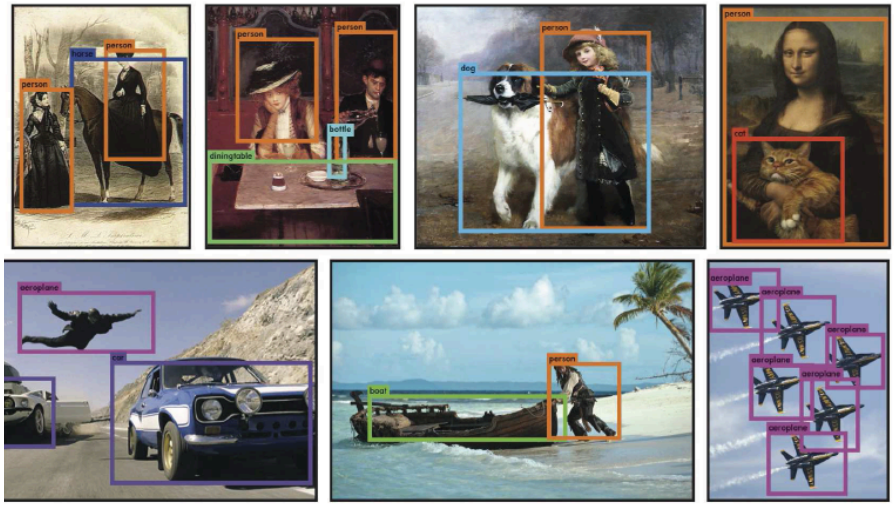


Figure 4 Results of YOLO detection

Object Recognition in Aerial Images Using Convolutional Neural Networks [10]

Radovic et al. focused on detecting anomalies in a transmission corridor from an Unmanned Aerial Vehicle (UAV) in flight.

Methodology:

The authors divided the process into three stages. First, raw data is captured by a UAV during flight. This is followed by real-time data processing by the on-board intelligence system. The final stage consists of a decision-making system that works based on the processed data. The process takes place in a matter of milliseconds." YOLO" (You Only Look Once) uses a single neural network to divide an image into regions, while predicting bounding boxes and probabilities for each region. These bounding boxes are weighted by the predicted probabilities. They approach object-detection as a tensor-regression problem. The size of the image entering the network needs to be in fixed format ($n \times m \times 3$, where the number 3 denotes 3 colour channels).

A recognition confidence of 84% was reached when a database containing satellite and UAV acquired images was used for the training purposes.

Results:

Results showed that the CNN (proposed network) was able to recognize airplane objects in the data set with 97.5% of accuracy (526 out of 540 airplane objects), while only 16 instances were incorrectly categorized (14 airplanes were not identified and 2 objects were wrongly identified). The CNN approach for object detection and classification in aerial images presented in this paper proved to be a highly accurate (97.8%) and efficient method although it required extensive computation. It proved to be more efficient compared to the traditionally employed machine learning algorithms with a detection and

classification accuracy of 84%.

- Lefebvre et al. [8] conducted image classification and image fusion between Sentinel and Landsat images based on evidence theory (Dempster–Shafer) for change detection in the cities of Prague and Rennes.
- Haas and Ban [11] took a combination of sentinel synthetic aperture radar and multi-spectral instrument data to detect changes in urban areas in Zürich's metropolitan area. Their methodology was comprised of canny edge detection, segmentation, object-based Support Vector Machine (SVM) classification and post-classification.
- In another study, Goldblatt et al.[12] used a Random Forest supervised image classifier in GEE on Landsat-8, Sentinel-1 and Sentinel-2 images as well as to classify changes in land use in Vietnam. Their work was done based on administrative cadastral data.

2.3 Proposed Solution

Our solution involves building a web application that detects and highlights changes or anomalies in a transmission line corridor using satellite imagery for monitoring the corridor's buffer zone.

2.4 Scope of Project

The system is meant to provide a means of inspecting a transmission corridor for officials of the national grid company GRIDCo using satellite image sources.

CHAPTER 3 - ENCROACHMENT DETECTION SYSTEM DESIGN AND DEVELOPMENT

3.1 Introduction

In this chapter, a full and thorough discussion of the methods used to accomplish the project's objectives are discussed. Details about the technological tools used in developing web application are fully discussed in Also this chapter.

3.2 System Overview and Operations

The web application serves as the main way for the user to access the system and perform the detection analysis. This was built using HTML, CSS, Python Django and JavaScript along with Bootstrap Framework. The application also uses the Google Maps JavaScript API. The Google Earth Engine (GEE) JavaScript API was used for the change detection system development. The Server application is responsible for hosting the application's database, the application's routes, the image segmentation system and the CNN model detection system. The server side was built or developed using Python Django framework. The interface of the system first allows the user to login to access the variety of options; check for detection within a tower range or view the whole Transmission line corridor. When selecting the detection option, the tower range image selected is fed as input to the server application for segmentation and processing. The communication between the server application and web application is done using Hyper Text Transfer Protocol (HTTP). Further discussion about the process is elaborated in the chapter. The GEE application is utilized for tracking anomalies in the corridor. It allows access to satellite imagery and geospatial information and processes it for visualization on the map. It has functionality for viewing the map with the marked towers and corridor buffer, obtaining area location as well as highlighting and filtering areas of change in the transmission corridor. This includes a dropdown for selecting tower locations for navigating over the pilot area. The interface allows the user to select a given tower location and view the area manually. There are also tools for highlighting areas on the map. The application was hosted as an earth engine user web application.

3.3 System Requirement and Specification

Functional Requirements

The encroachment detection system was designed and created to meet the following functional requirement:

1. Allow users to login to the web application
2. Allow users to select tower location for viewing purpose
3. Allow users to select tower images for detection of encroachment
4. Allow system to segment captured image into smaller section for model classification
5. Allow system to unify the segmented, classified images to a single picture
6. Allow users to store the image into the database for future reference

7. Allow users to perform real-time tracking of the corridor selected

Non-functional Requirements:

The non-functional requirements of the detection system are that the application developed shall:

1. Be easy to understand and interact with
2. Be user friendly
3. Have a quick response time
4. Be reliable and available always
5. Be secured and well maintained

Also, other requirements were needed in order to develop the system. These were highlighted below

Specifications from utilities (GRIDCo):

- Geolocation coordinates of towers (KMZ data)
- Distance between transmission lines
- Length of lines
- Transmission line expanse
- Right of way corridor width
- Tower Coding Names Convention

Requirements for Satellite Image repository (Google Earth Engine):

The source of satellite imagery should have:

- Recent images of a given location preferably from the same sensor
- Satellite images with little or low cloud cover of the area specified.
- Same spatial resolution of images of a given area over time
- Atmospherically corrected data taken at similar time of day for correct classifications and same visualization over a given area.

Specifications for Satellite Image Change Detection:

The change detection system should be able to:

1. Get the location at which a change or possible encroachment has taken place for a given geographical location.
2. Find the amount and the kind of change that has taken place.
3. Provide visualization of areas of change with intensity values

3.4 Engineering Design Solution

The web application entails the image selected going through the process of segmentation and classification, before being unified together to show the encroached area. After that, a notification is sent

to the necessary authorities showing the tower range in which the encroachment was detected. We implement a change detection model, for real time tracking of changes within the corridor, that performs image differencing on Sentinel-2 surface reflectance images (the highest quality atmospherically corrected images publicly available), to detect changes in a given area. For change detection of a more recent and an older image, a change of 0 means no change, positive or negative indicate change. The flow diagram and architectural diagram depicting the whole process are shown below

Flow Diagram

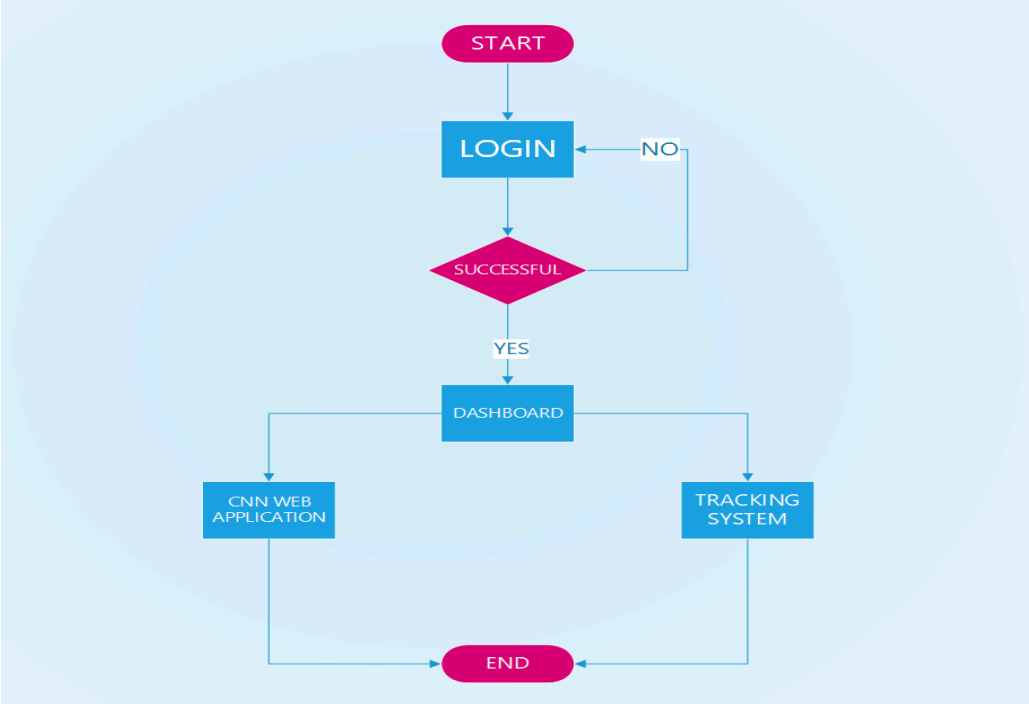


Figure 5 Operational flow detection system

Architectural diagram of Detection and Tracking System

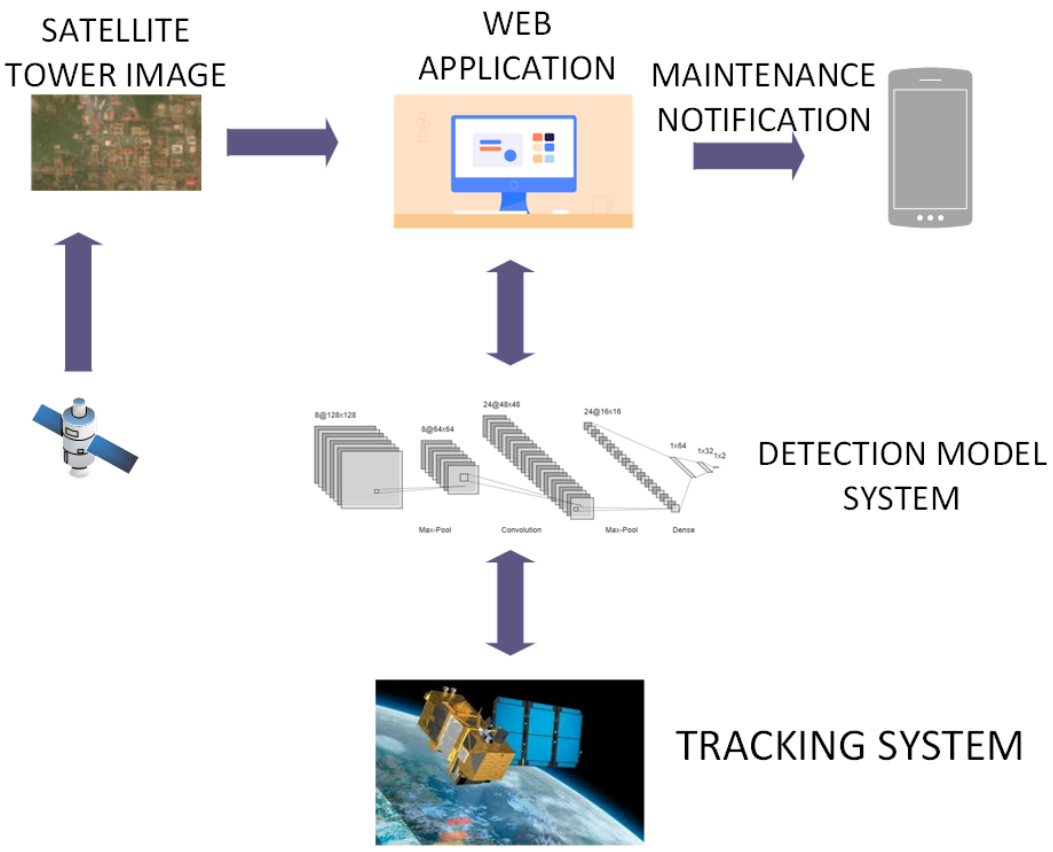


Figure 6 Detection and Tracking System

Block diagram of Detection and Tracking System

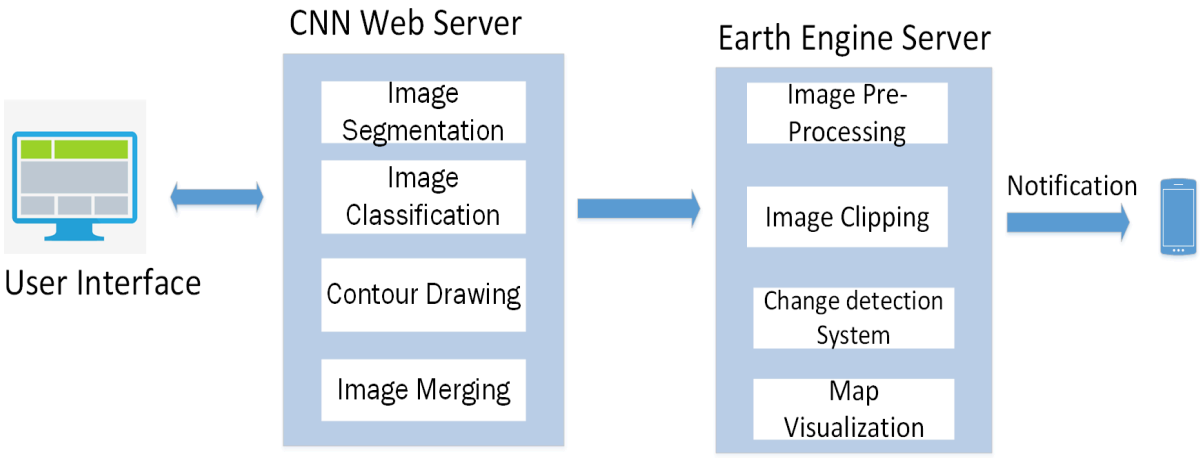


Figure 7 Block diagram of System

3.5 Data Gathering

In gathering data for the system, it consisted of two parts, which were collection of data for the convolutional neural network model and data for the tower transmission lines and locations

This consisted of data used to train the model and were collected from various sources. The sources included online dataset repository Kaggle which had variation of the training dataset but these were not sufficient as the images varied from that of most housing structures in Ghana. Thus, a satellite view of houses in Ghana were taken from Google Maps and cropped to attain the right size suitable to train the CNN model.

Figure 8 Sample of images of dataset obtained from Kaggle website

Figure 9 Sample of images of dataset obtained from Google Maps

The transmission line information was obtained from GRIDCo as a keyhole mark-up language file (KMZ file), which is a file type compatible with Google Earth. The tower locations for the corridor was given by GRIDCO in MS excel spreadsheet specifying the Northings and Eastings. When opened in Google Earth Pro the KMZ file highlights the transmission corridor for viewing on the base map. A representation of the transmission line is highlighted in red inFigure 10.

Figure 10 Visualization of a KMZ file in Google Earth Pro.

A3BSP - Volta_L3

POWERLINES_GH:Voltage	161
POWERLINES_GH:Situation	Existing
POWERLINES_GH:C design	2
POWERLINES_GH:C inst	2
POWERLINES_GH:ID1	GH
POWERLINES_GH:ID2	GH
POWERLINES_GH:km	13.2
POWERLINES_GH:Year	2000
POWERLINES_GH:EXT1	A3BSP
POWERLINES_GH:EXT2	Volta
POWERLINES_GH:multipart	0
POWERLINES_GH:NUML	3
POWERLINES_GH:NUMCKT	1
POWERLINES_GH:CIRCUIT	V7H
POWERLINES_GH:NAMEC	A3BSP - Volta_L3_C1
POWERLINES_GH:NAMEF	161 A3BSP - Volta_L3_C1
POWERLINES_GH:TYPE	Line
POWERLINES_GH:CKT NAME	Achimota - Kpong
POWERLINES_GH:PARTCKT	1
POWERLINES_GH:BEGIN	2000/01/01
POWERLINES_GH:END	2050/01/01
POWERLINES_GH:check_alph	1

Figure 11 View of metadata information for a KMZ file

A3BSP - Volta_L3.xlsx [Protected View]

	A	B	C	D
1				
2		NORTHINGS	EASTINGS	
3		625607.67	820424.95	
4		625641.03	820506.37	
5		625760.94	820788.65	
6		625873.06	821072.15	
7		625996.19	821381.48	
8		626109.88	821648.8	
9		626241.98	821979.47	
10		626372.3	822301.2	
11		626488.56	822570.17	
12		626600.05	822851.34	
13		626742.38	823208.48	
14		626873.95	823520.11	
15		627003.87	823840.89	
16		627138.96	824171.72	
17		627267.03	824479.79	
18		627400.81	824931.18	
19		627542.68	825161.17	
20		627694.68	825522.77	
21		627821.39	825841.65	

Figure 12 Snapshot of MS Excel tower locations

The tower locations for the pilot corridor were given as northings and eastings (UTM Coordinates) for each tower. However, we require the tower position coordinates in decimal latitude and longitude format. The latitude and longitude coordinates were derived from the UTM coordinates using the UTM converter [14] online tool and stored as CSV files. The format for a sample output file after conversion is shown in Table 2 Sample tower location Information Table 2

Table 2 Sample tower location Information

northing	easting	longitude	latitude	zone
630479.4	167874.2	-5.998092024	5.696148	30
630443.5	832331.2	-5.74208E-05	5.695815	30
630336.7	832124.2	-0.001929184	5.69486	30
630262.8	831939.4	-0.003599732	5.694201	30
630262.7	831939.6	-0.003597661	5.6942	30
630170.8	831715.6	-0.005622131	5.69338	30
630096.8	831533.6	-0.007267154	5.69272	30
630005.7	831312.2	-0.009268313	5.691907	30
629906.6	831071.9	-0.01143984	5.691023	30
629790.3	830783.7	-0.014044068	5.689986	30
629683	830512.3	-0.016497079	5.68903	30
629591.7	830300.1	-0.018415084	5.688214	30
629496.1	830063.3	-0.020555048	5.687362	30
629397.5	829822.3	-0.022733303	5.686482	30

3.6 Assumptions for System Design

We define anomalies in the corridor as an unwanted object or settlement as well as a relatively large difference in an image of the same area over a given time period or in a transmission corridor.

A snapshot of the transmission corridor from Google Maps satellite view was taken at a given time for use in this project for the convolutional neural network. We assume the images of the selected corridor to be recent and sufficient for our application's needs. Furthermore, we assume changes detected due to cloud cover can be cross checked by the user to eliminate false detections.

3.7 Detection Model System

This section entails the CNN model design, the testing and training phase along with its performance and evaluation

3.7.1 Dataset and Processing

The image taken from the google maps API goes through a series of steps before being fed into the AI model developed for detection and also another process for unification occurs after the detection process is done. These processes are:

1. **Image Segmentation**: It first goes through the process of segmentation, where the image is divided into smaller segments of the original and stored as individual images. This process is performed in order to feed each of the regions of the map to the model for detection. This also prevents the wrong classification of the images as minute sizes produces better results when fed into the model. This process is achieved by dividing the image array into specified rows and columns and assigned unique names for identification. The rows and columns values used in the division process chosen were of 100 by 100 respectively. This was chosen as it helps the model in obtaining the optimum result for the classification. After classification, if a structure is detected for encroachment, a bounding box is drawn around it to show the area of encroachment.

2. **Image Unification:** After each segmented image is fed, classified and bounding boxes drawn around the, the process of unification takes place, where all the segmented images are brought together to form the original image. This process is done by keeping track of each of the images along with the unique names associated with it, which is used when recombining the segmented images to form the original. The number of columns and rows were also specified in order to attain the right image size the complete unified image contains the detected contours indicating encroachment if any exists.

A flow diagram indicating the process is shown in Figure 13.

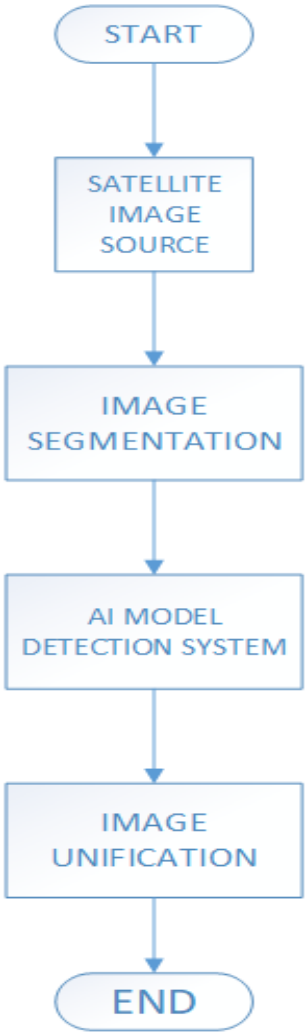


Figure 13 Image processing flow structure

3.7.2 Convolutional Neural Network

This section of the system involved training and using an AI model with Convolutional Neural Network. Convolutional Neural Network CNN), which is a class of artificial neural network, is a Deep learning Algorithm that takes an input image and performs various calculations and functions to extract the features out of the image, which is then used to differentiate one from another.

The CNN architecture has input, convolutional layer, pooling layer and connected layers. Different architecture types are used depending on the design.

In developing the Convolutional Neural Network model for the system, various parameters such as: input image size, kernel size, padding, number of outputs, and others were considered in order to train and use a model suitable for the system's objective.

The CNN architecture, as shown in Figure 14, was built with a neural system having of 128 neurons representing the features extracted from the images with an output of 2 cells representing the two output classes(House structure and Land).

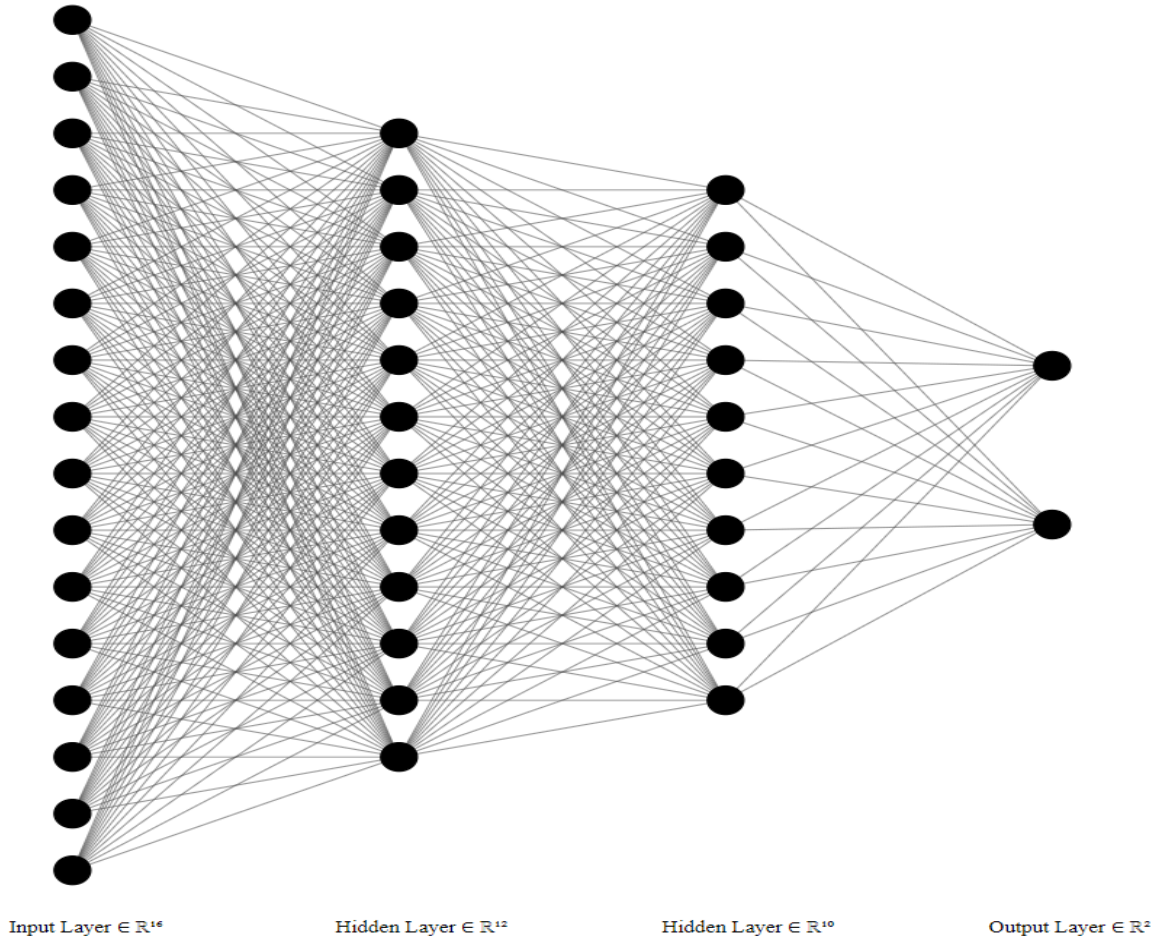


Figure 14 Convolutional Neural Network Architecture

During inference, the features obtained from the tower images are passed into the model. Based on the weights learnt during training, the model classifies the input features into one of the two outputs. The flow diagram summarizes the diagnostic procedure

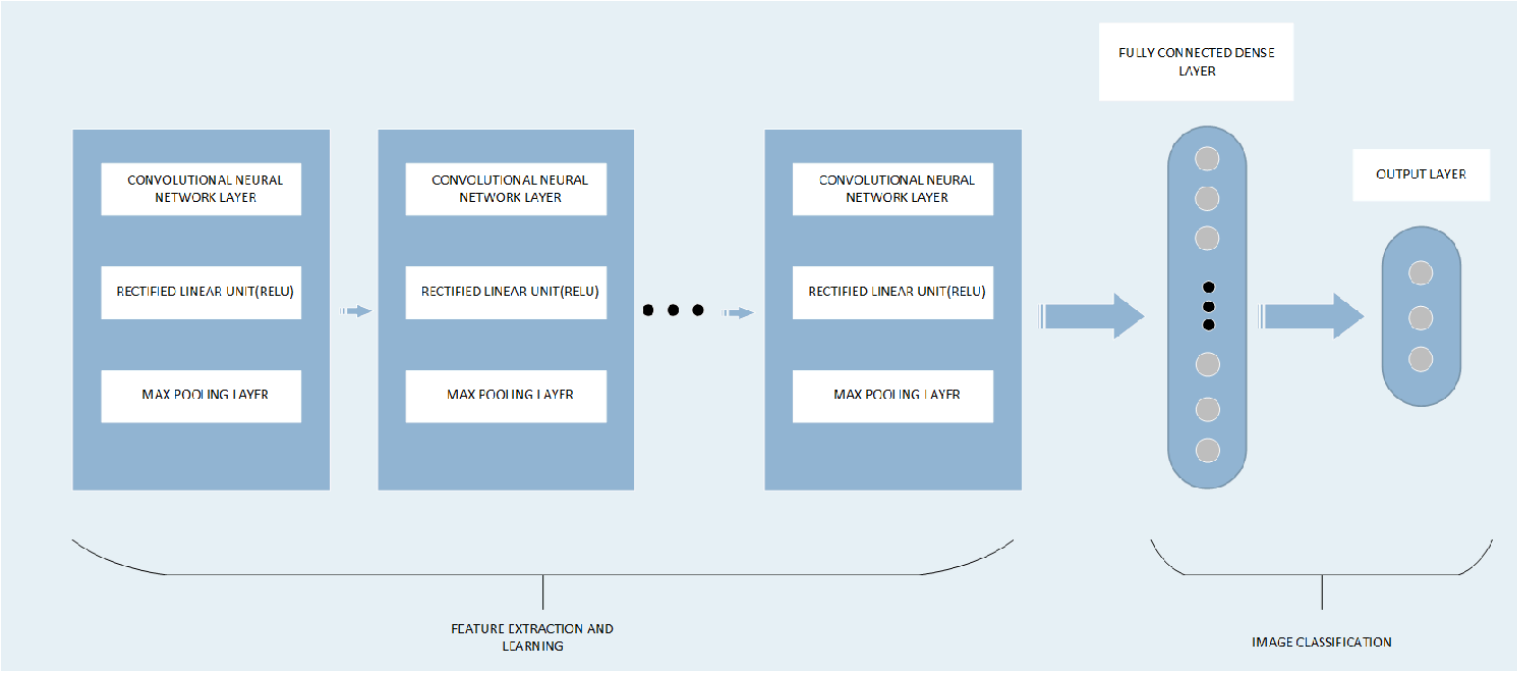


Figure 15 Flow of the CNN model design

Dataset Training: In developing the model, a total of 3600 images were collected for training and testing of the model. The table below shows a summary of how the images were divided for the training and testing of the model

Table 3 Dataset information

Categories\image partitioning	**TRAINING**	**TESTING**	**TOTAL**
HOUSES	1500	300	1800
LAND	1500	300	1800
TOTAL	3000	600	3600

Input Image: The input image size used during the training was 128 x 128-pixel RGB (Red, Green Blue) images. In order to train the model to be able to recognize the categories mentioned, the above dimension was chosen so as to view the selected categories sufficiently and not to make the categories too blurry for the classification model. Also, the size of the total images used in training the model was taken into account

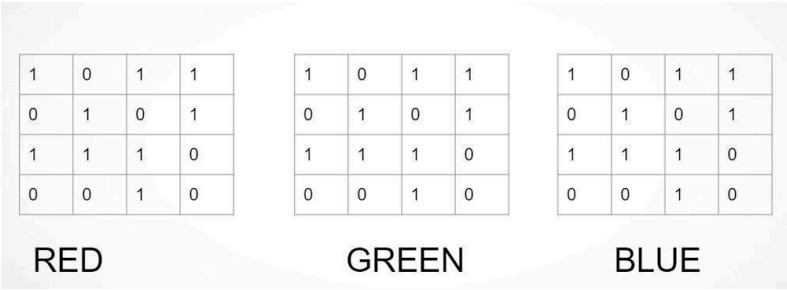


Figure 16

Kernel Size: After the input image size, the kernel size, which is a filter (in matrix form) for the feature extraction of the image is chosen for high level feature extraction. The kernel size of 3 by 3 matrix form was chosen in correlation with the image size given.

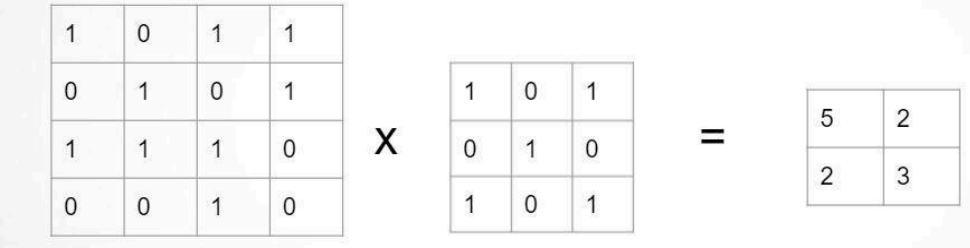


Figure 17 Kernel Operation

Padding: The padding of value being same is chosen. This ensures that the output size of the model is the same as that of the input, whether half padding or full padding is applied to get it.

Activation Function: The activation function used in the CNN model was rectified linear activation function or ReLU for short and the SoftMax.:

> ***Rectified linear:*** This is a piecewise linear function that will output the input directly if it is positive, otherwise, it will output zero. It has become the default activation function for many types of neural networks model because, it is quicker to train and often produces better performance [13].
>
> ***SoftMax***: Activation function used in the final layer was that of the SoftMax or the normalized exponential function. The SoftMax function is used as the activation function in the output layer

of neural network models that predict a multinomial probability distribution [14] . The equation of the SoftMax is shown below

EQUATION FOR SOFTMAX ACTIVATION FUNCTION

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \ldots\ldots$$

$$\sigma = softmax$$

$$\vec{z} = input\ vector$$

$$e^{z_i} = standard\ exponential\ function\ for\ input\ vector$$

$$K = number\ of\ classes\ in\ the\ multi\text{-}class\ classifier$$

$$e^{z_j} = standard\ exponential\ funciton\ for\ output\ vector$$

Loss function: Loss is nothing more than a Neural Net prediction error that calculates the gradients. Gradients are also utilized to update the Neural Net's weights. This was how we trained a Neural Network. The loss function used in training the model was categorical loss function. This is a multi-class classification loss function that gives the output between a range of 0 to 1, based on the categories [15].

Outputs: The output of the CNN model is of 2 classes, namely: House and Bare Land. The two outputs were chosen due to the nature of the geographical area of the selected region for testing.

Table 4 Hyper Parameters of CNN system

HYPER PARAMETERS	
Input Size	[128x128x3]
Learning Rate	0.001 - 0.01
Hidden Layers	2 - 4
Neurons per layer	16 -128
Activation Function	ReLU, SoftMax
Loss Function	Binary Cross entropy, Categorical Cross entropy
Output Unit	2

3.7.3 CNN Model Training and Testing

After the specifications for the parameters of the CNN model were specified, the hidden layers were varied in order to obtain the best result for the system. In training the system, a call-back function is

triggered in case the training of the model start depreciating in accuracy after a certain number of epochs.

The table below show how the neurons in each layer were changed and its result along with it

Table 5 Three hidden layers with learning rate of 0.1

Hidden Layers = 3 Learning rate = 0.1			
Layer 1	Layer 2	Layer3	Accuracy
32	64	128	51.27%
16	32	64	50.43%
8	16	32	52.32%

Table 6 Three hidden layers with learning rate of 0.01

Hidden Layers = 3 Learning rate = 0.01			
Layer 1	Layer 2	Layer3	Accuracy
32	64	128	87.23%
16	32	64	85.97%
8	16	32	85.30%

Table 7 Three hidden layers with learning rate of 0.001

Hidden Layers = 3 Learning rate = 0.001			
Layer 1	Layer 2	Layer3	Accuracy
32	64	128	97.27%
16	32	64	95.46%
8	16	32	94.56%

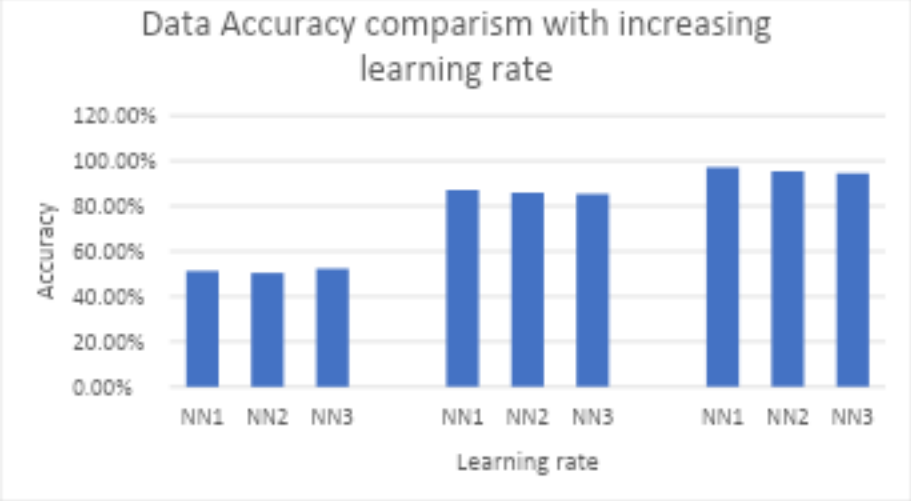


Figure 18 Data Accuracy comparison

Model Selection

The model with three hidden layers and a learning rate of 0.001 was chosen after hyper-parameter tuning and optimization. The following table summarizes the model's entire specifications:

Selected hyper parameters for Model design	
Hidden Layers	3
Neurons	[128, 64, 32]
Learning rate	0.001
Accuracy	97.27
Kernel Size	3x3
Input Shape	128x128x3
Pooling size	3
Strides	3

Figure 19

After the selected hyper parameters were used in training the CNN model, the summary containing all parameters and layers in a summarized form is shown in Figure 19.

```
Model: "sequential_22"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_95 (Conv2D)           (None, 63, 63, 64)        832
_________________________________________________________________
max_pooling2d_68 (MaxPooling (None, 31, 31, 64)        0
_________________________________________________________________
conv2d_96 (Conv2D)           (None, 30, 30, 128)       32896
_________________________________________________________________
max_pooling2d_69 (MaxPooling (None, 15, 15, 128)       0
_________________________________________________________________
conv2d_97 (Conv2D)           (None, 14, 14, 256)       131328
_________________________________________________________________
max_pooling2d_70 (MaxPooling (None, 7, 7, 256)         0
_________________________________________________________________
conv2d_98 (Conv2D)           (None, 6, 6, 128)         131200
_________________________________________________________________
flatten_16 (Flatten)         (None, 4608)              0
_________________________________________________________________
dense_51 (Dense)             (None, 128)               589952
_________________________________________________________________
dense_52 (Dense)             (None, 2)                 258
=================================================================
Total params: 886,466
Trainable params: 886,466
Non-trainable params: 0
```

Figure 19 Model summary

After the compilation values were chosen, the various parameters for the fitting process such as the number of epochs, validation data and training data were input and trained. Given the loss parameter of Categorical Cross entropy, the optimizer was varied and the model trained repeated with various image inputs from the dataset.

The epoch training process with the optimizer of "Adam" with loss being "categorical cross entropy" is shown in the image below

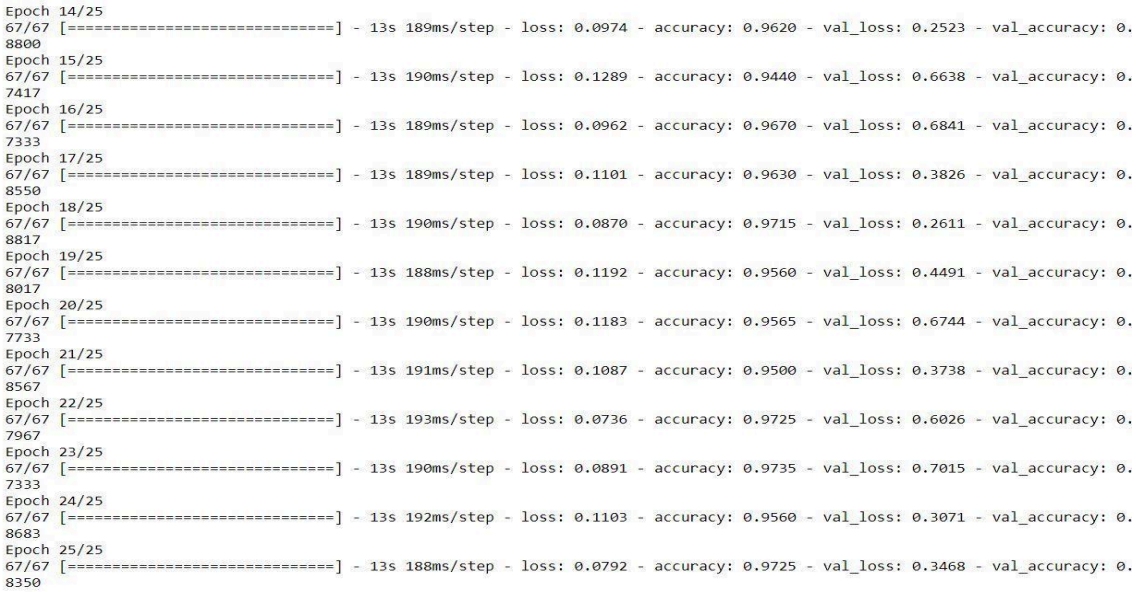


Figure 20 Adam training process

Another epoch training process with an optimizer of "Adamax" and loss of "category cross entropy" is shown below:

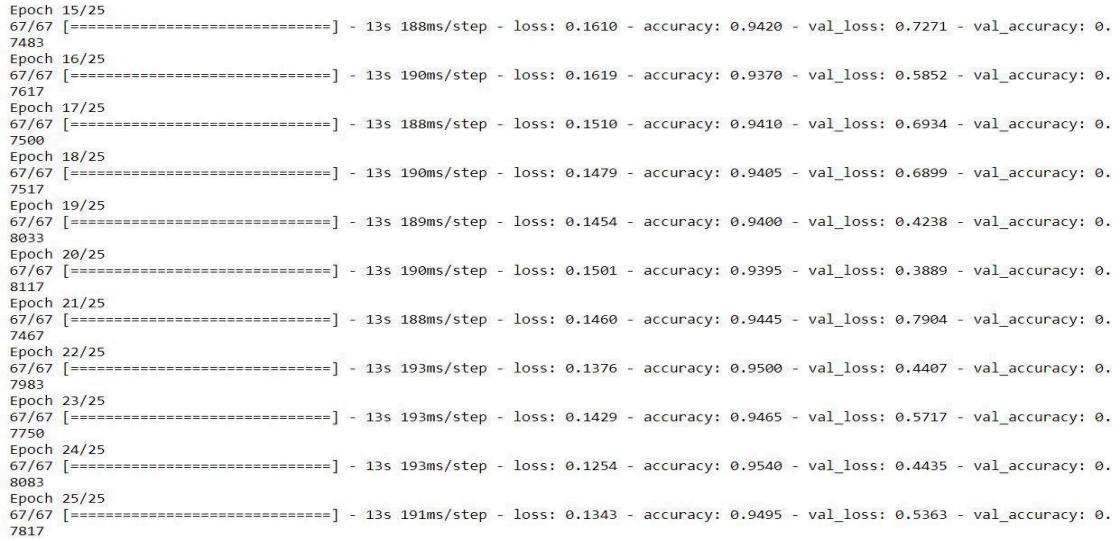


Figure 21 Adamax training process

Another epoch training process selecting the optimizer as "RMSPROP" with the loss function being "categorical cross entropy" is also shown below

```
Epoch 14/25
67/67 [==============================] - 13s 190ms/step - loss: 0.1620 - accuracy: 0.9375 - val_loss: 0.4197 - val_accuracy: 0.
8133
Epoch 15/25
67/67 [==============================] - 13s 190ms/step - loss: 0.1420 - accuracy: 0.9450 - val_loss: 0.4716 - val_accuracy: 0.
7533
Epoch 16/25
67/67 [==============================] - 13s 192ms/step - loss: 0.1413 - accuracy: 0.9450 - val_loss: 0.4205 - val_accuracy: 0.
8017loss: 0.1415 - accuracy: 0.
Epoch 17/25
67/67 [==============================] - 13s 194ms/step - loss: 0.1620 - accuracy: 0.9405 - val_loss: 0.2997 - val_accuracy: 0.
9000
Epoch 18/25
67/67 [==============================] - 13s 198ms/step - loss: 0.1322 - accuracy: 0.9490 - val_loss: 0.4113 - val_accuracy: 0.
8183
Epoch 19/25
67/67 [==============================] - 14s 204ms/step - loss: 0.1387 - accuracy: 0.9465 - val_loss: 0.3544 - val_accuracy: 0.
8567
Epoch 20/25
67/67 [==============================] - 15s 218ms/step - loss: 0.1322 - accuracy: 0.9490 - val_loss: 0.3689 - val_accuracy: 0.
8550
Epoch 21/25
67/67 [==============================] - 14s 211ms/step - loss: 0.1229 - accuracy: 0.9565 - val_loss: 0.3591 - val_accuracy: 0.
8433
Epoch 22/25
67/67 [==============================] - 13s 198ms/step - loss: 0.1168 - accuracy: 0.9550 - val_loss: 0.3512 - val_accuracy: 0.
8483
Epoch 23/25
67/67 [==============================] - 13s 198ms/step - loss: 0.1383 - accuracy: 0.9535 - val_loss: 0.4363 - val_accuracy: 0.
7983
Epoch 24/25
67/67 [==============================] - 13s 193ms/step - loss: 0.1138 - accuracy: 0.9590 - val_loss: 0.3362 - val_accuracy: 0.
8733
Epoch 25/25
67/67 [==============================] - 13s 191ms/step - loss: 0.1418 - accuracy: 0.9505 - val_loss: 0.3747 - val_accuracy: 0.
8333
```

Figure 22 RMSPROP training process

3.7.4 Model Performance and Evaluation

This section shows the performance of each of the various trained model along with its loss function.

The model was tested with various images to show its model performance in predicting the category of the input image

Sample testing of the system with images showing the classification category for each in Figure 23

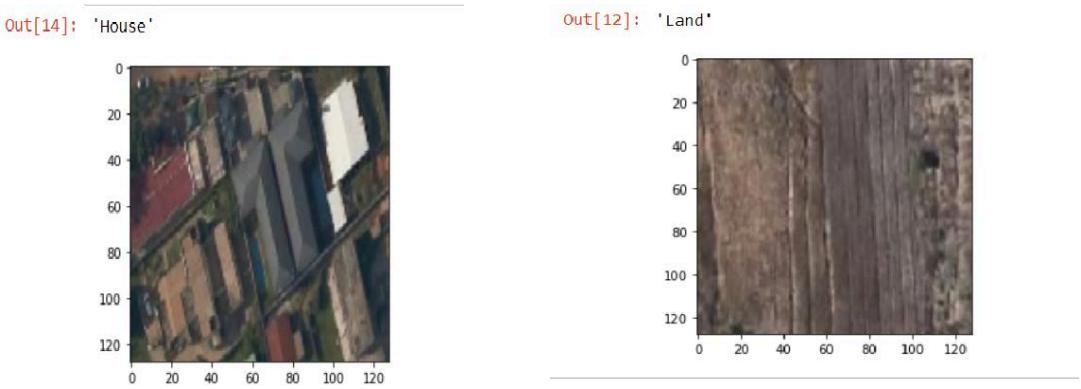


Figure 23 Model Performance

The evaluation of the model after training and testing is shown in Table 8 indicating the Test loss and the Test Accuracy after passing the test data through the model for evaluation.

Table 8 Model Evaluation

MODEL EVALUATION	
TEST LOSS	0.11260360
TEST ACCURACY	0.96833333

Below shows each of the optimizers model accuracy and model loss diagrams

ADAMAX

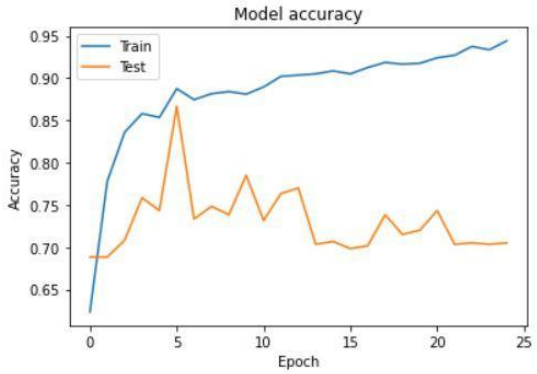


Figure 24 Adamax Model Accuracy

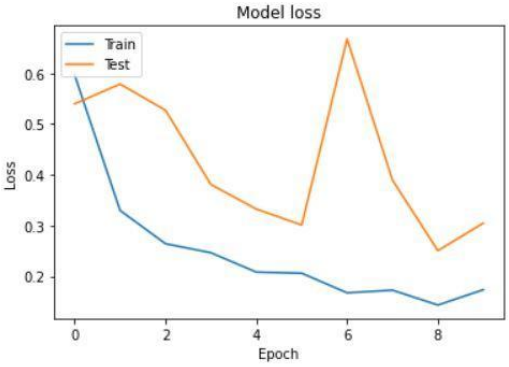


Figure 25 Adamax Model Loss

ADAM

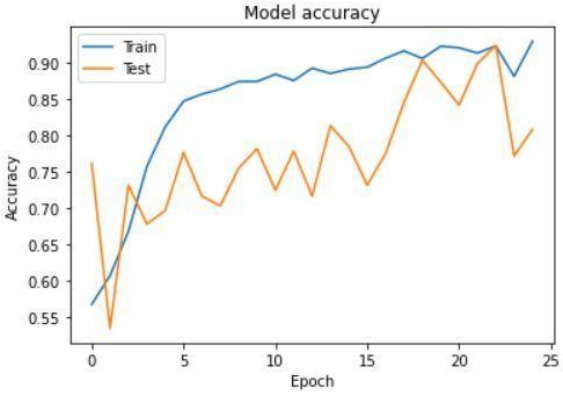


Figure 26 Model Accuracy

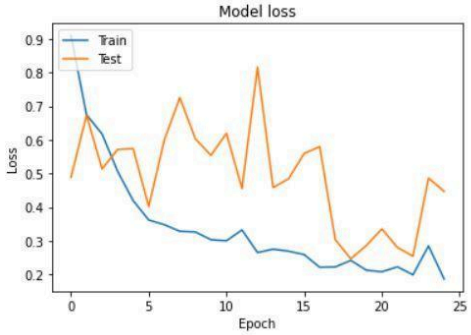


Figure 27 Adam Model Loss

1. RMSPROP

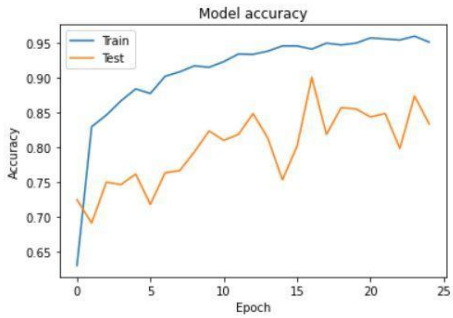


Figure 28 RMSprop Model accuracy

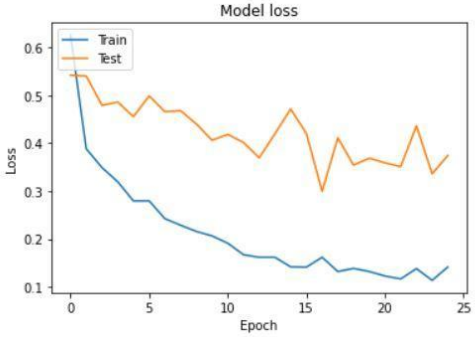


Figure 29 RMSProp Model Loss

Based on the model accuracy and loss, the chosen optimizer for the system was the Adam optimizer.

3.8 Change Detection System Using Google Earth Engine

Further to monitor any incident of encroachment the GEE change detection system was added as a means of tracking encroachment in the corridor. We implemented a change detection model that performs image differencing on Sentinel-2 satellite images, the highest quality atmospherically corrected images publicly available at 10m spatial resolution.

3.8.1 Change detection system specifications

In performing change detection, we look at certain considerations for the images to be compared. The images preferably should be:

- Taken from the same sensor
- Have the same spatial resolution
- Taken at similar time of day
- Atmospherically corrected and cloud free.

This is to minimize changes due to data characteristics we are not interested in such as cloud cover.

3.8.2 Satellite Image Pre-processing

In considering changes in satellite images due to urban growth, changes on the landscape were detected as changes in the spectral values of pixels to detect encroachment in the transmission corridor's right-of-way. The Sentinel-2 satellite imagery has various bands for computation such as the red, green, blue and Near-Infrared (B4, B3, B2, B8) bands. A sample of image band values as seen in the GEE inspector for a point on a satellite composite is shown in Figure 30.

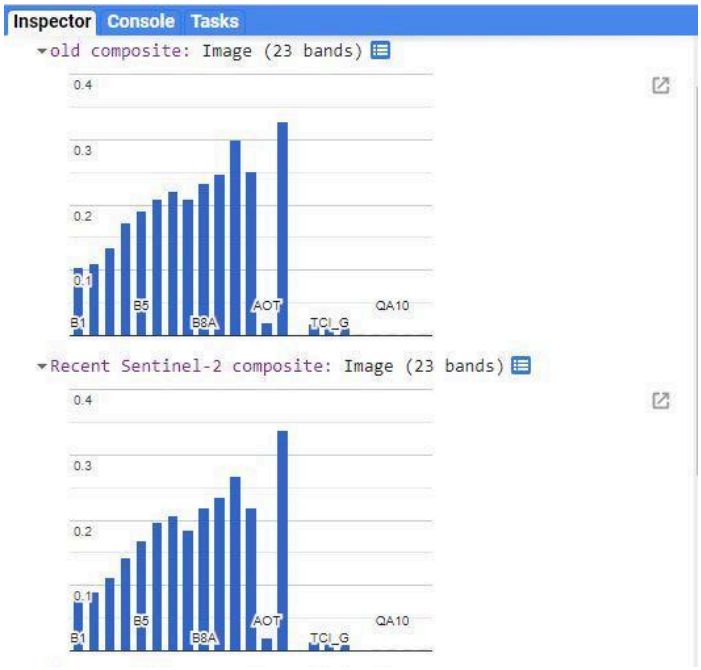


Figure 30 Comparison of older and recent image band values

In order to obtain the satellite images for comparison we selected images in a chosen time period whose bounds intersect the geometry (manually drawn outline) covering our area of interest from the Sentinel-2 collection. We used a time interval because selecting a single instance of time cannot be guaranteed to have a suitable image of the given area. The image collection is filtered to remove images with 'CLOUDY PIXEL PERCENTAGE' greater than a given percentage value and cloud masked. We used a median reducer to obtain a single image composite of the collection to obtain a single image and then clip the images to the corridor's buffer. We performed this process to obtain the recent and older images we will be using for comparison in the change detection model. The flow diagram of the system is shown below

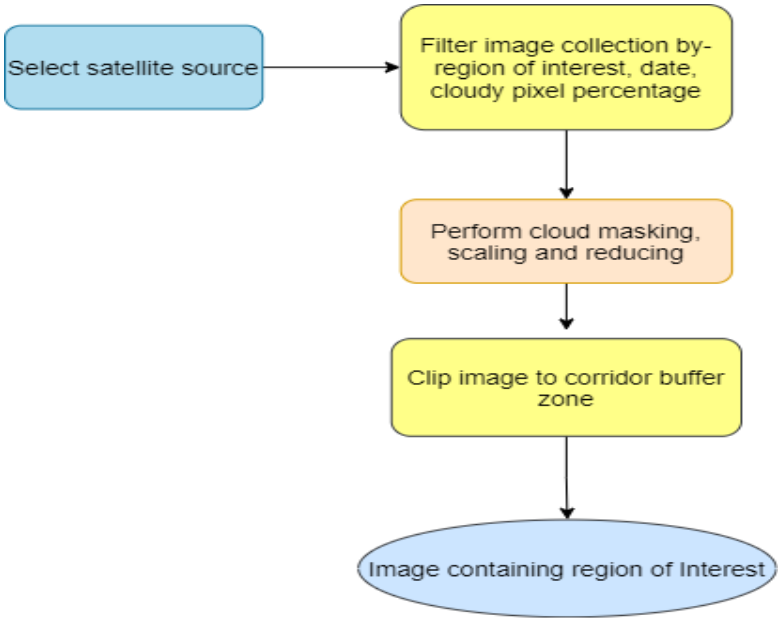


Figure 31 Satellite image pre-processing flow diagram

3.8.3 Change Detection System Overview

For change detection of a more recent and an older image, a change of 0 means no change, positive or negative values indicate change. Figure 32 shows the result of image differencing (-0.2582) for a point on the map.

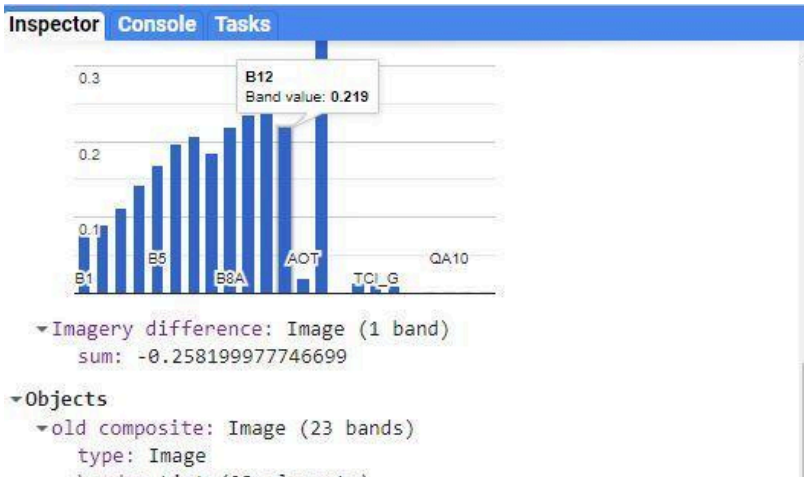


Figure 32 Image differencing chart results

We set a threshold value to differentiate changes less than a given magnitude and eliminate changes less than that threshold. Additionally, we eliminate detections due to agricultural change from the results using the Normalized Difference Vegetation Index (NDVI) since vegetation change is not of interest in this study and add the results to the map layer. The process is illustrated in Figure 33.

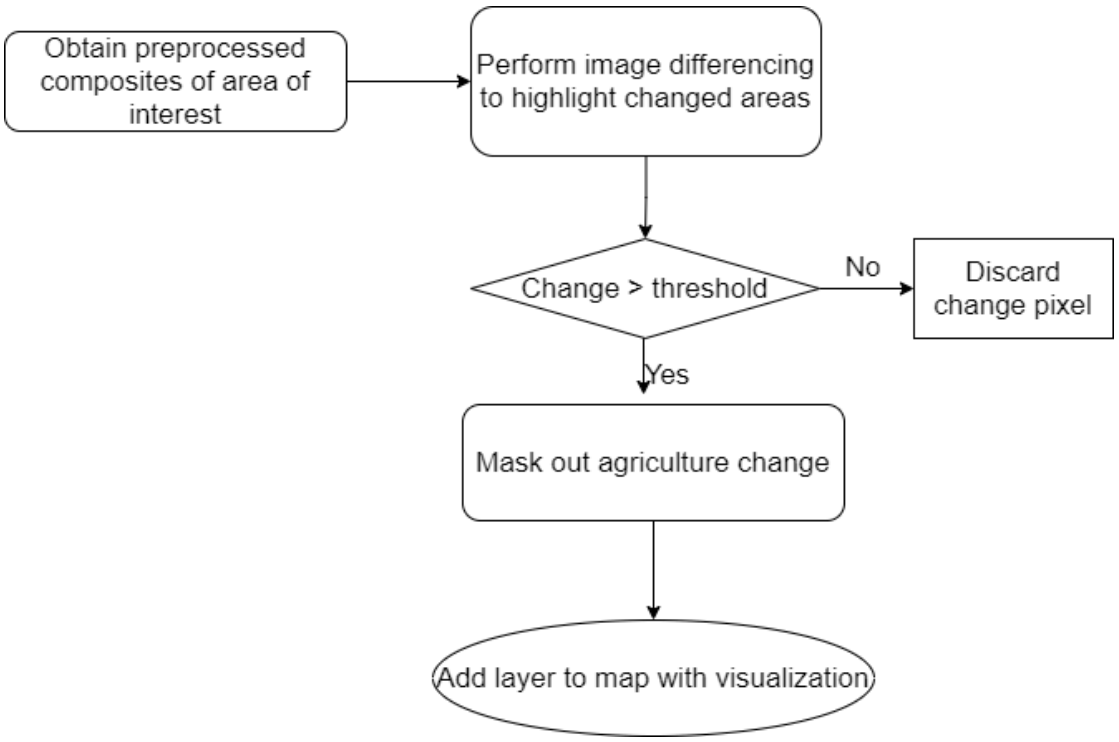


Figure 33 Change detection system flow chart

3.9 Application Software Development

This section involves the design and development of the system's web application and its Database Design

3.9.1 Database Design

The database consists of the tables used within the whole web application. These tables include:

1. The User Table
2. Corridor Lines
3. Tower Names
4. Tower Images

These tables are interlinked to each other as shown in the ER diagram below.

s

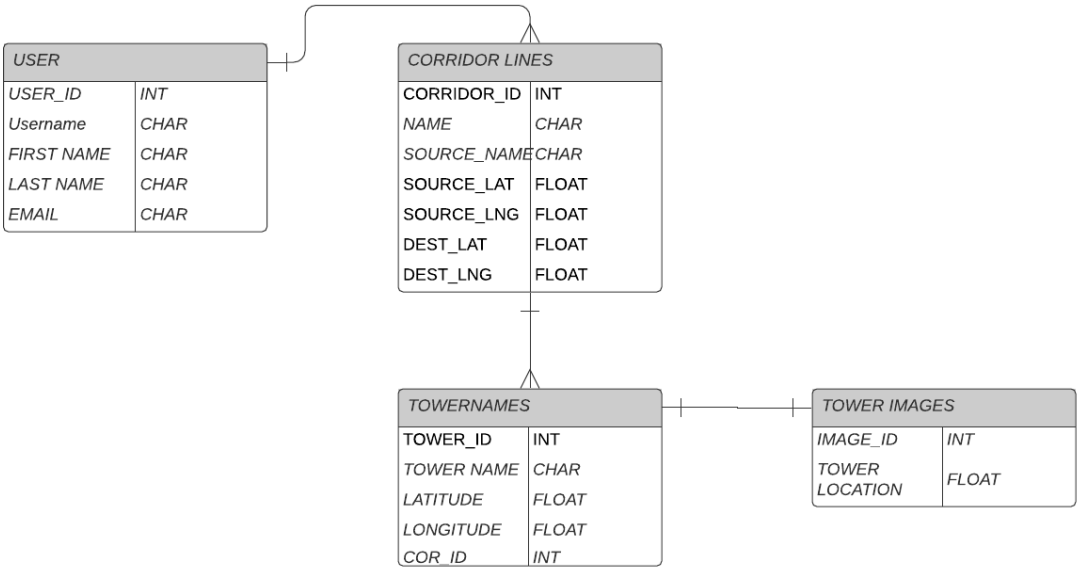


Figure 34 ER diagram

3.9.2 Web Application Design and Development

1. **The Tracking Change web application**: The map view with the change detection layers, tower points and dropdown of tower locations was built using the Google Earth Engine online JavaScript code editor.

2. **The Detection System web application**: This involves building a web application that uses the CNN model trained to detect encroachment within the line of corridor. This is done by first taking the image and segmenting it into smaller pieces for the model. After the segmentation, each image is fed into the model for classification, and based on its classification, bounding boxes are drawn on each of the segmented images to denote the encroached area. After that the segmented images are stitched together with the drawn bounding boxes showing the encroached area. These drawn bounding boxes then show the type of encroachment based on the prediction made. A flow diagram showing the thorough process is shown in Figure 36 Flow diagram of detection system:

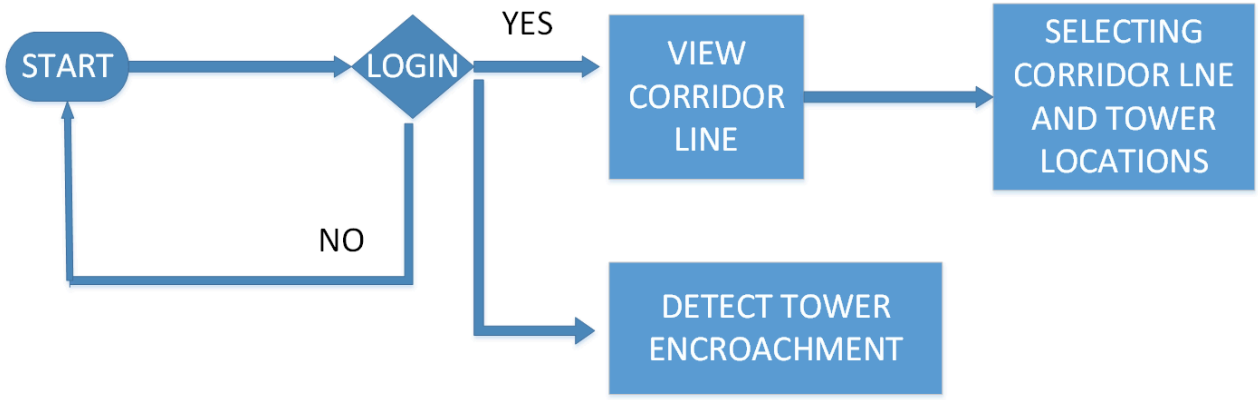


Figure 35 Flow diagram of detection system

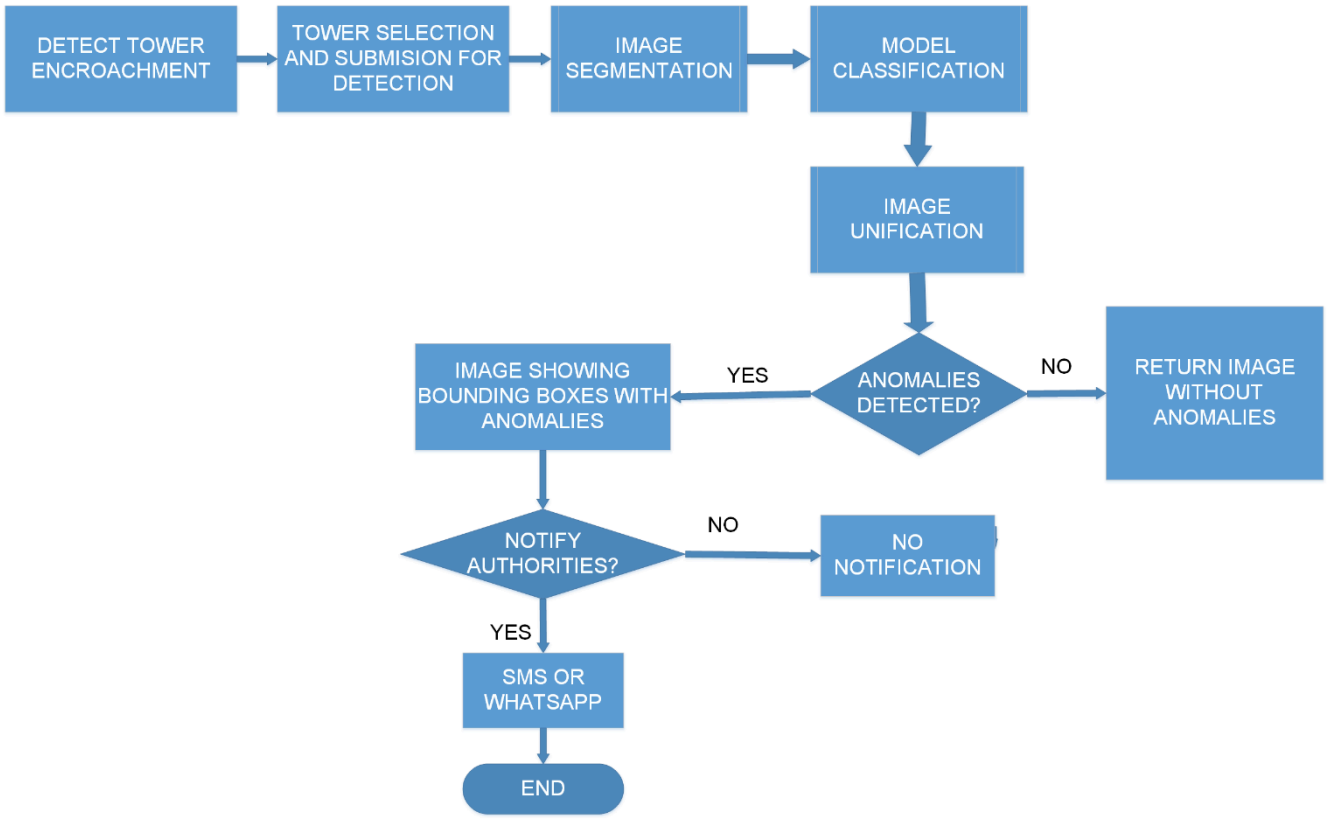


Figure 36 Flow diagram of detection system cont'd

Use Case Scenario

The figure below depicts the use case diagram for the application, which displays the many ways in which a user can interact with the application for detection and tracking of encroachment within the corridor.

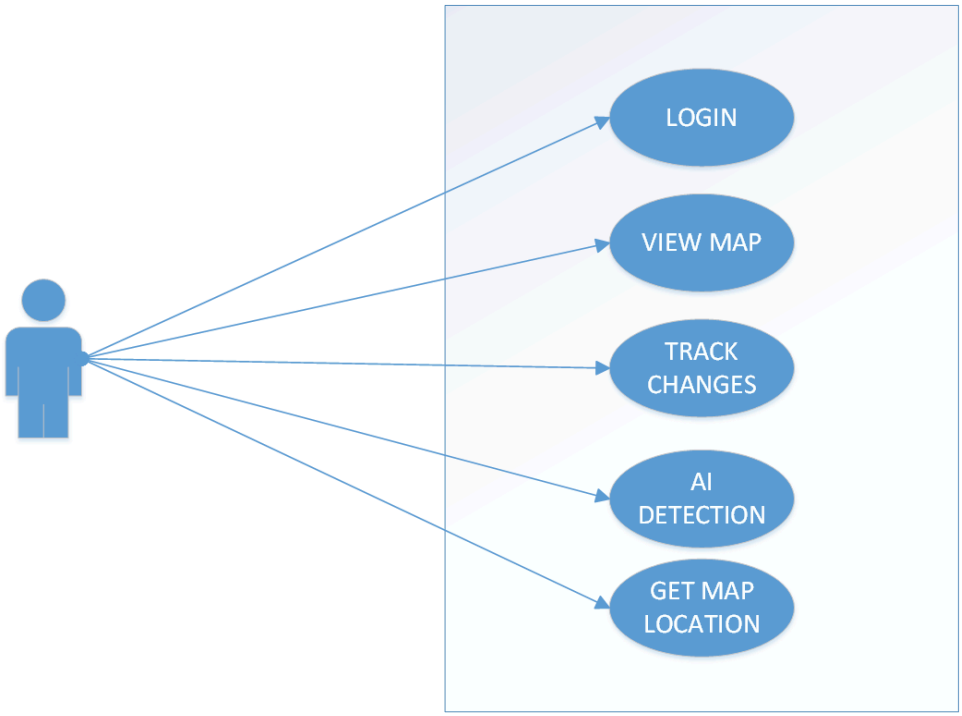


Figure 37 Use case diagram

3.10 Development Tools and Material Requirements

1. Google Earth Engine[16]

A cloud-based platform for planetary-scale geospatial analysis that brings Google's massive computational capabilities to bear on a variety of high-impact societal issues including deforestation, water management, climate monitoring and environmental protection. It provides a platform for querying, visualizing and analysing data from publicly available satellites, including Sentinel-2, as well as a wide range of datasets through an Application Programming Interface (API) which we interact with using the JavaScript programming language.

2. Google Earth engine code editor [17]

Web based Integrated Development Environment for JavaScript which provides a git-based script editor, map display for visualizing geospatial datasets, geometry map tools, view of results of the JavaScript code in the console and allows you to perform techniques such as machine learning and image classification. It served as a tool for viewing the transmission corridor and also for viewing the intermediate result of the queries on the Sentinel-2 data. This was used to build the web-based application with tower locations as a dropdown in the satellite view for the user to select and view the corridor.

3. Google Earth Pro

A mapping application for viewing an interactive map of the surface of the earth. Also useful for creating overlays and user defined points, lines and polygons on the map. It was used for visual representation of KMZ files and for marking locations of transmission towers in a corridor.

4. UTM to Latitude and longitude converter [18]

   This online tool was used to convert the UTM coordinates (Northings and Eastings) provided by GRIDCo for the tower locations to latitude and longitude geographic coordinates in decimal degrees.

5. Notepad++

This text editor was used to edit generated csv files from the UTM to Latitude and Longitude converter to take the longitude and latitude information.

6. Visual Studio Code

The CNN model was built and trained using this desktop application. It provided an interface for viewing and executing jupyter notebooks as well as python scripts.

7. Python libraries: OpenCV, TensorFlow, Matplotlib, Keras

These were used to build and train the CNN model for classification and detection of anomalies from high resolution images.

8. Google Chrome Browser
   The GEE system development and the detection system tests were carried out using Google Chrome Version 92.0.4515.159 (Official Build) (64-bit).
9. Twilio API

   Twilio Messaging is an API to send and receive SMS, MMS, OTT messages globally. It uses intelligent sending features to ensure messages reliably reach end users wherever they are.

CHAPTER 4 - IMPLEMENTATION AND TESTING

4.1 Introduction

This section shows the processes for the testing and implementation of the Encroachment Detection System built and the results from the system operation and testing.

The testing involved three phases namely:

1. Image Segmentation and Unification
2. The Encroachment Detection system
3. The GEE change detection system

4.2 System Implementation Process

This section of the chapter gives details about the CNN model integrated with the web application created, along with the image processing that occurs.

This section entails selecting a pilot area for the testing in each of the three phases of the system. In implementing the testing of the system, a laptop of Lenovo ideapad 3 was used in testing the various phases of the implemented system.

Image Segmentation

In implementing the image segmentation and unification process, a pilot region around east Legon, near the A3BSP supply station was selected. Its location being latitude of 5.65269 and longitude of -0.105402 as source and a latitude of 5.65355 and a longitude of -0.103302 as the destination.

Also, in order to assure the correct image is obtained correctly, an image with a size of 1267 by 671 pixels was used during the testing. The segmented images, segmented to a size of 100 by 100-pixel images, sizes mostly range from 80 to 84 images being obtained from the images to be segmented

The image is represented in Figure 38.

Figure 38 Selected image for segmentation

The picture below shows the previous mentioned image being segmented into various parts

Figure 39 Segmented Images

Image Unification

After the segmented images are fed into the model for detection, bounding boxes/contours around the images classified as structures are drawn on the images and unified to get back the whole tower region.

The Figure 40 shows the images with drawn contours on them:

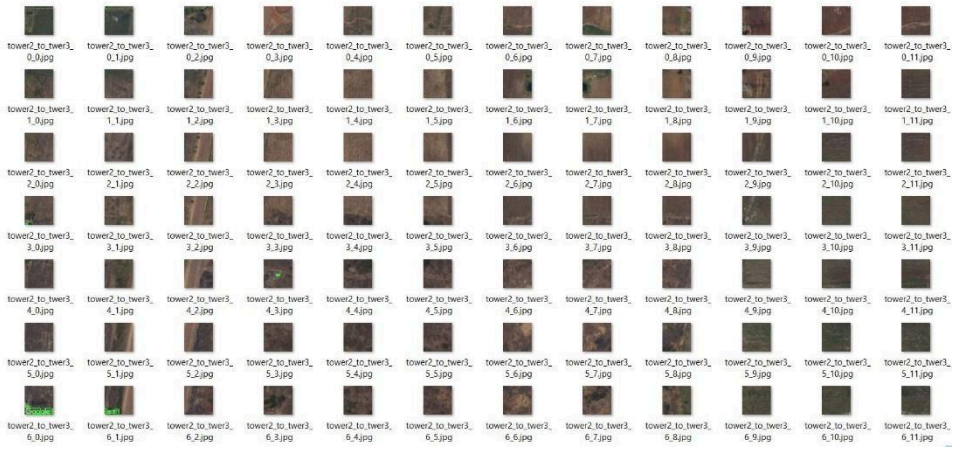


Figure 40 Segmented images with contours drawn

After the contours or bounding boxes are drawn, the images are unified again with the areas showing the area of encroachment, if any exist on the selected area:

4.2.1 Encroachment Detection Application System

This entails the user interacting with the web application to perform various actions on the web app in order for the AI model to detect the encroached area.

Login

In order to access the system, the login user interface allows the user to give authentication credentials in the form of a username and a password. It gives the user fields for username and password and a login button that sends an HTTP POST request when clicked. This makes a request to the server with the validated username and password data procedure for authentication. Upon logging in successfully, the user is routed to the dash board, whereas the user is prompted if the login credentials are wrong and

remains on the login page. The Figure 41 Login interface shows the interface of the web application login interface

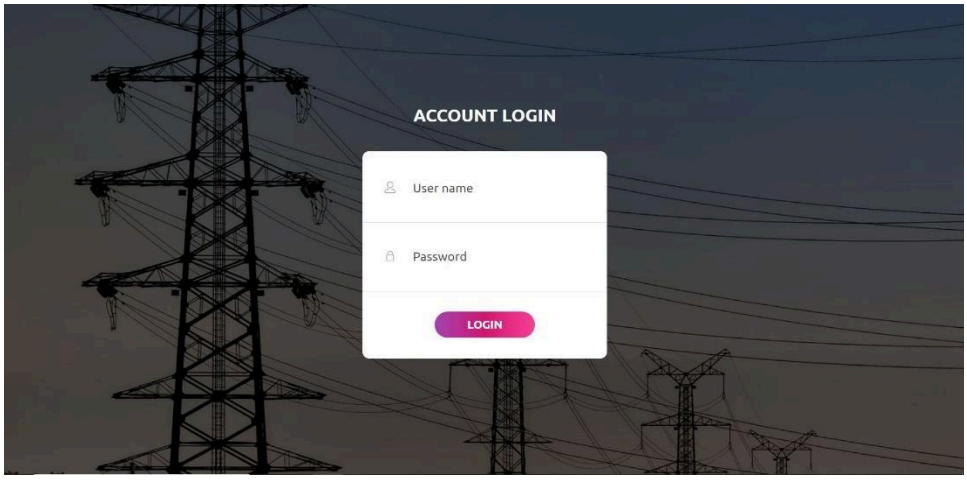


Figure 41 Login interface

Dashboard Interface

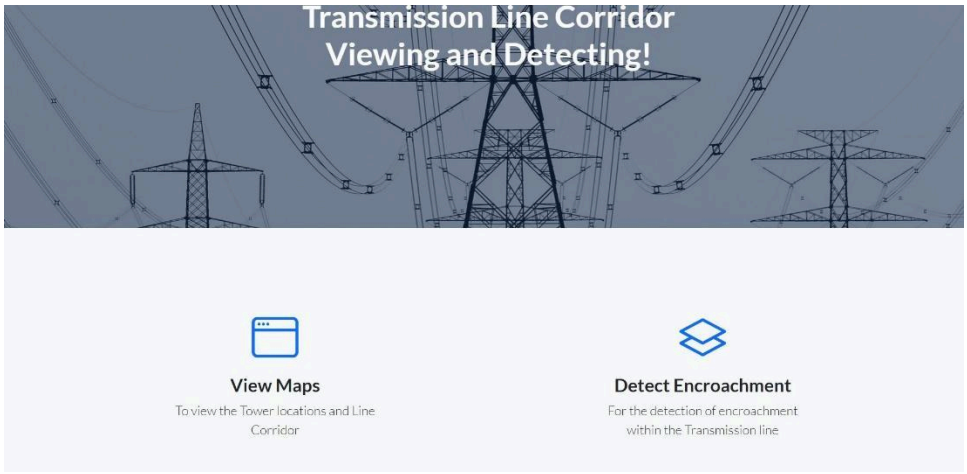


Figure 42 Dashboard

Tower Selection and Detection

In performing the detection of encroachment within the transmission line corridor, a case area study was selected from the A3BSP to VOLTA corridor, since it contained various areas which were both heavily and lightly encroached. The selected tower regions for the case were tower 4 to tower 5 and tower 13 to tower 14, having its coordinates in Table 9 Tower 4 to Tower 5 Coordinates) and Table 10 Tower 13 Tower 14 Coordinates respectively. The images used in the testing of the Detection system were the current and updated Google Earth Pro images (updated on 3rd March, 2021).

Table 9 Tower 4 to Tower 5 Coordinates

TOWER 4 TO TOWER 5	
Source Latitude	5.65355

Source Longitude	-0.103302
Destination Latitude	5.65446
Destination Longitude	-0.101051
Zone	30

Table 10 Tower 13 Tower 14 Coordinates

TOWER 13 TO TOWER 14	
Source Latitude	5.68139
Source Longitude	-0.0353668
Destination Latitude	5.68226
Destination Longitude	-0.0331829
Zone	30

The select tab contains the tower locations used to call the images from the database, in which, allows the user to select the towers to be fed into the model for detection of encroachment. After the selection of the towers, the image is submitted for detection using the CNN model built.

DETECTION OF TOWER4 TO TOWER5

The first test conducted were of the tower 4 to tower 5 which is shown in Figure 43 tower4 to tower5

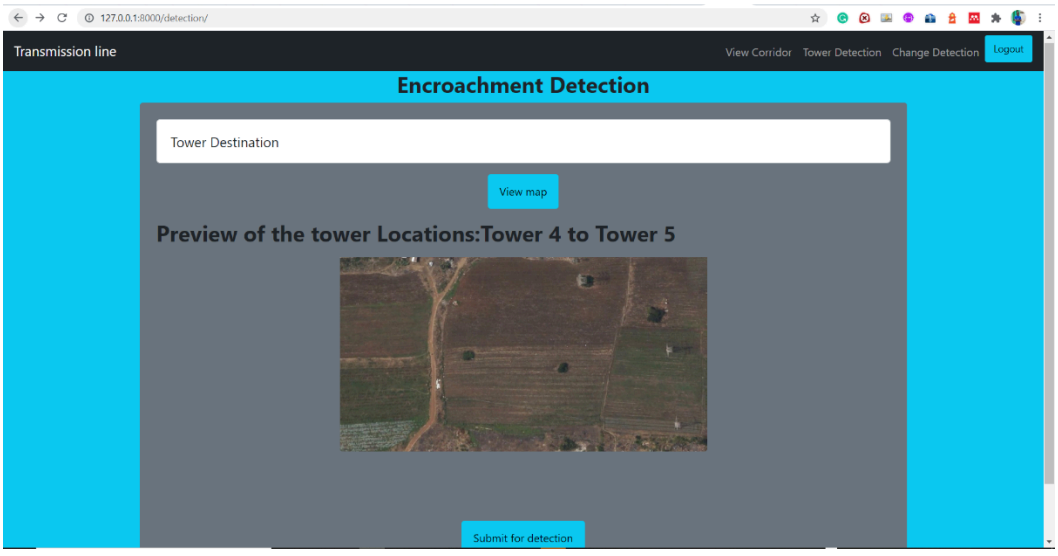


Figure 43 tower4 to tower5

After the sample tower was passed through the Detection system, the detected structures for encroached area were four and bounding boxes were drawn around them to indicate their position in the tower region.

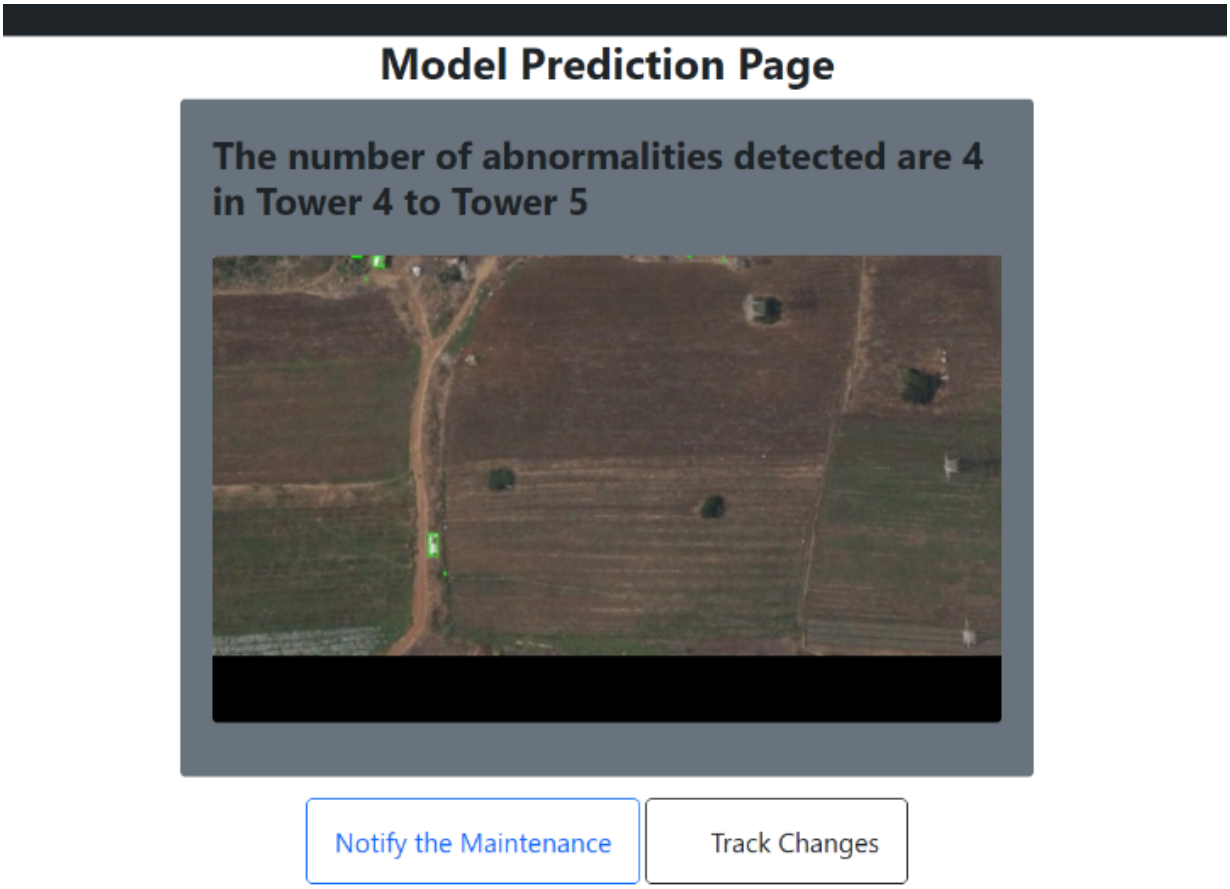


Figure 44 Tower 4 to tower5 after detection

In detecting the encroached area, the notification button for notifying the maintenance team is shown as when clicked, it sends the tower locations of the area of encroachment to the maintenance team via WhatsApp notification. When the WhatsApp notification is sent, a success message is shown to the user for successful message. The success message is shown in Figure 45 Notification Status

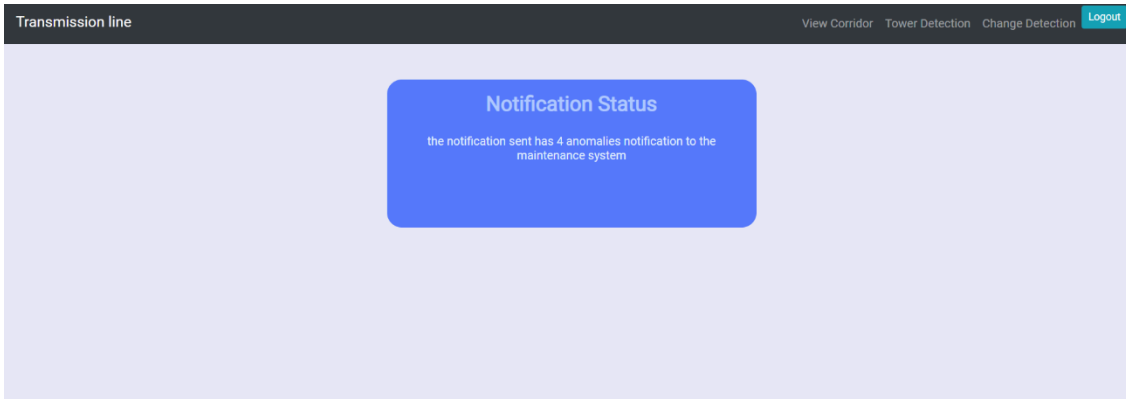


Figure 45 Notification Status

The WhatsApp notification, sent using the Twilio API, is shown in Figure 46 WhatsApp Notification. The message received contains the tower locations containing the encroachment and also, the number of encroachments within that corridor were indicated in the message sent.

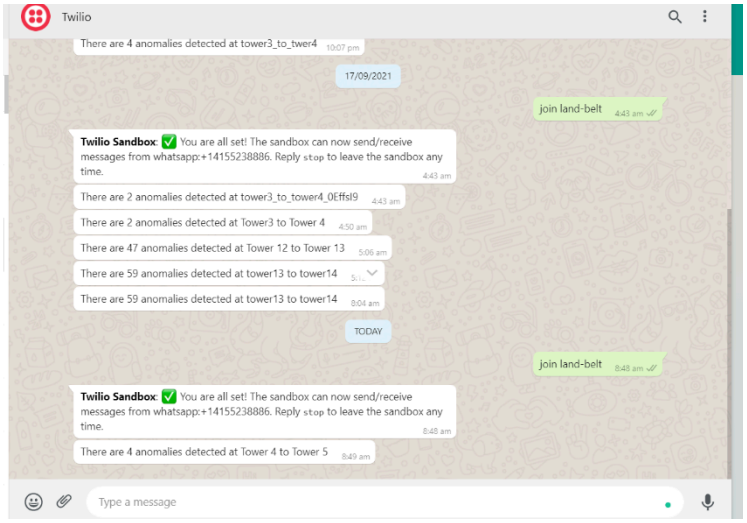


Figure 46 WhatsApp Notification

Thus, the system was able to showcase its detection ability in the corridor selected in Figure 43 tower4 to tower5 and was able to detect all the areas having encroached areas within the ROW (Right of Way) corridor of the tower location.

DETECTION OF TOWER 13 TO TOWER 14

The second test area used for the testing of the detection system was of Legon to Volta Line 1, being tower 13 to tower 14.

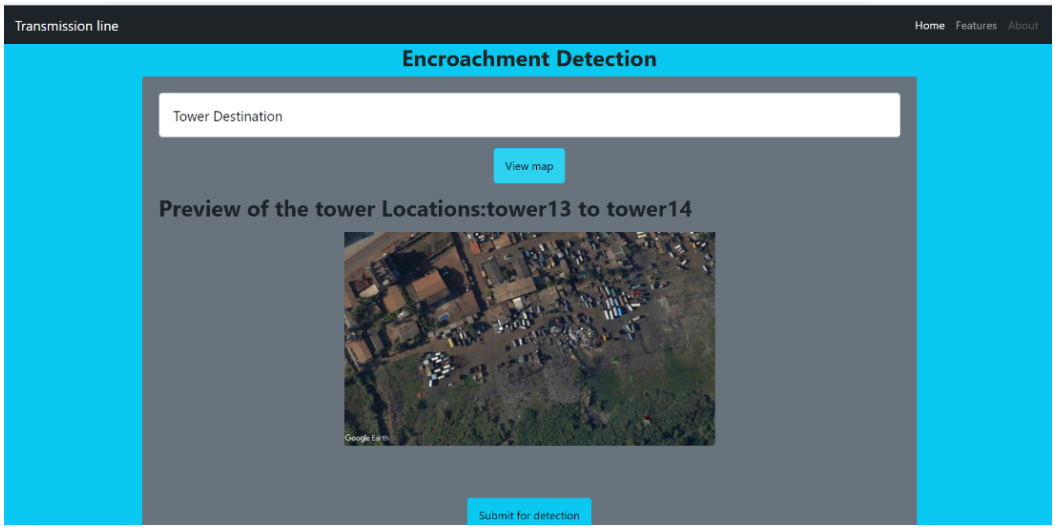


Figure 47 Tower 13 to Tower 14

After the image containing the tower region is selected, it is passed through the detection system via the "submit for detection" button, after which it is subjected to the image processing and classification. The Figure 48 Tower13 to Tower14 after Detection showcase the area after the system checked for encroachment.

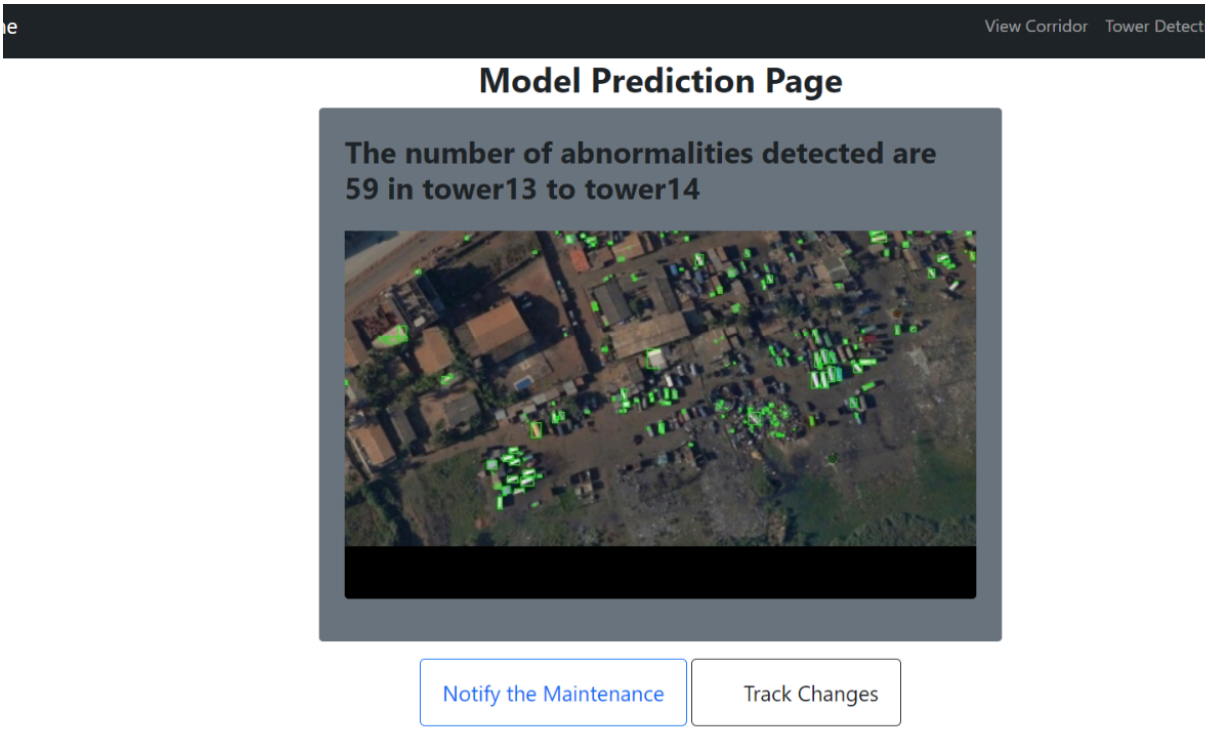


Figure 48 Tower13 to Tower14 after Detection

The notification button for contacting the maintenance team is displayed when an encroached area is detected, and when hit, it transmits the tower locations of the encroached area to the maintenance team via WhatsApp notification. When a WhatsApp notification is delivered, the user is shown with a success message. Figure 49 Tower13 to Tower14 Notification status shows the success notification status.

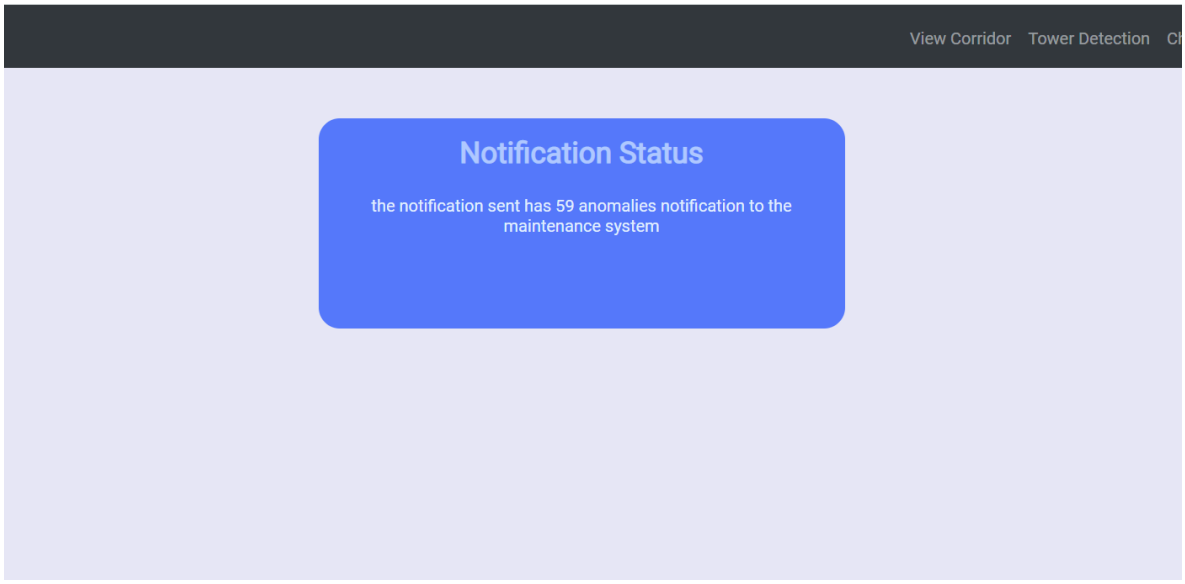


Figure 49 Tower13 to Tower14 Notification status

The WhatsApp notification, sent using the Twilio API, is shown in Figure 50 Tower13 to Tower14 WhatsApp notification. The message received contains the tower locations containing the encroachment and also, the number of encroachments within that corridor were indicated in the message sent.

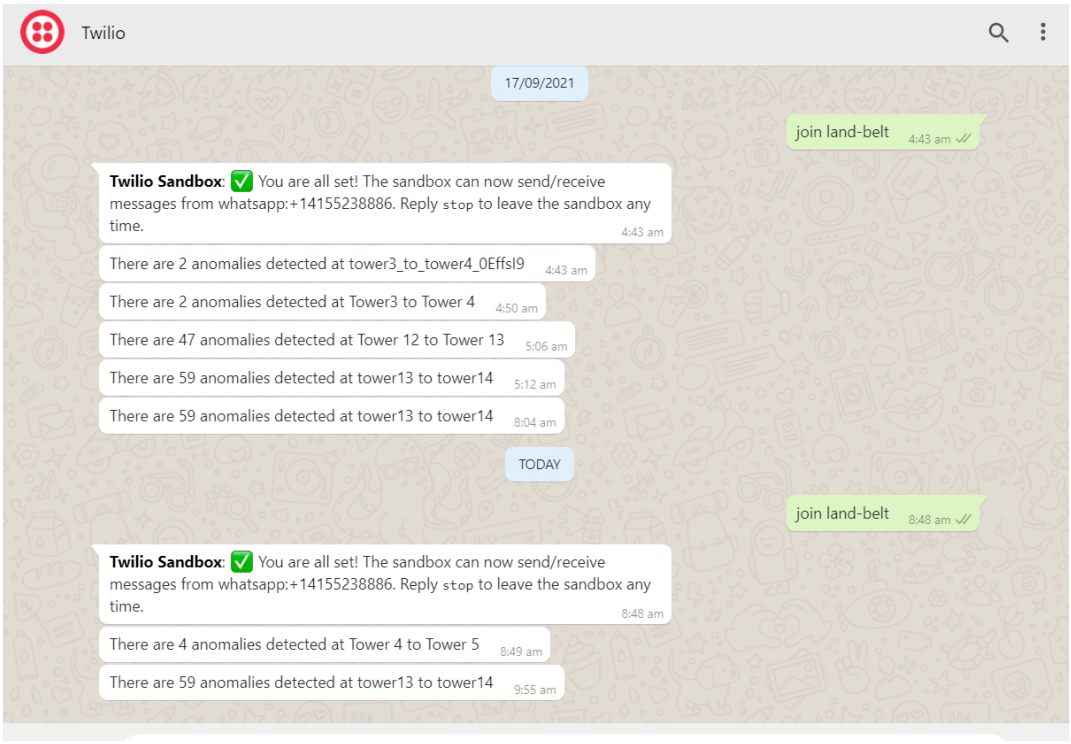


Figure 50 Tower13 to Tower14 WhatsApp notification

Thus, the system was able to demonstrate its detection capacity in the selected area in Figure 47 Tower 13 to Tower 14 corridor, detecting all encroached regions within the ROW (Right of Way) corridor of the tower position.

Twilio API System

Twilio Messaging is an API to send and receive SMS, MMS, OTT messages globally. It uses intelligent sending features to ensure messages reliably reach end users wherever they are. The Simple Plan service implemented in the system price was $0.75 per message sent. The Twilio API is used in sending the WhatsApp messages from the Detection system to the end user (maintenance team). The codes below show

```
account_sid = 'your account-sid'
auth_token = 'your auth-token'
client = Client(account_sid, auth_token)


def SMS():
        imageSMS = client.messages.create(
                                messaging_service_sid='your account-sid’,
                                body='message you want to send',
                                to='mobile number to receive messages'
                            )
```

Google Earth Engine Change Detection System

Prototype Implementation Process

The change detection system was built using Google Earth Engine (GEE) using the GEE JavaScript API. A pilot corridor consisting of 50 towers from Tema Community 18 to Community 9 for testing the system. We used the Google Chrome browser for building the application in the earth engine code editor and also for testing functionality of the web application. The application was run on a Lenovo laptop with 8 Gigabytes of RAM on a Windows 10 operating system. The pre-processed satellite images as outlined in section 3.8.2 were used in the change detection system. The locations (latitudes and longitudes) of towers in the corridor were stored as a 2-dimensional array and superimposed as geometry points on the base map in GEE. The path of the transmission line was added as a line on the base map. The system prototype with the superimposed towers and corridor line is shown in Figure 51.

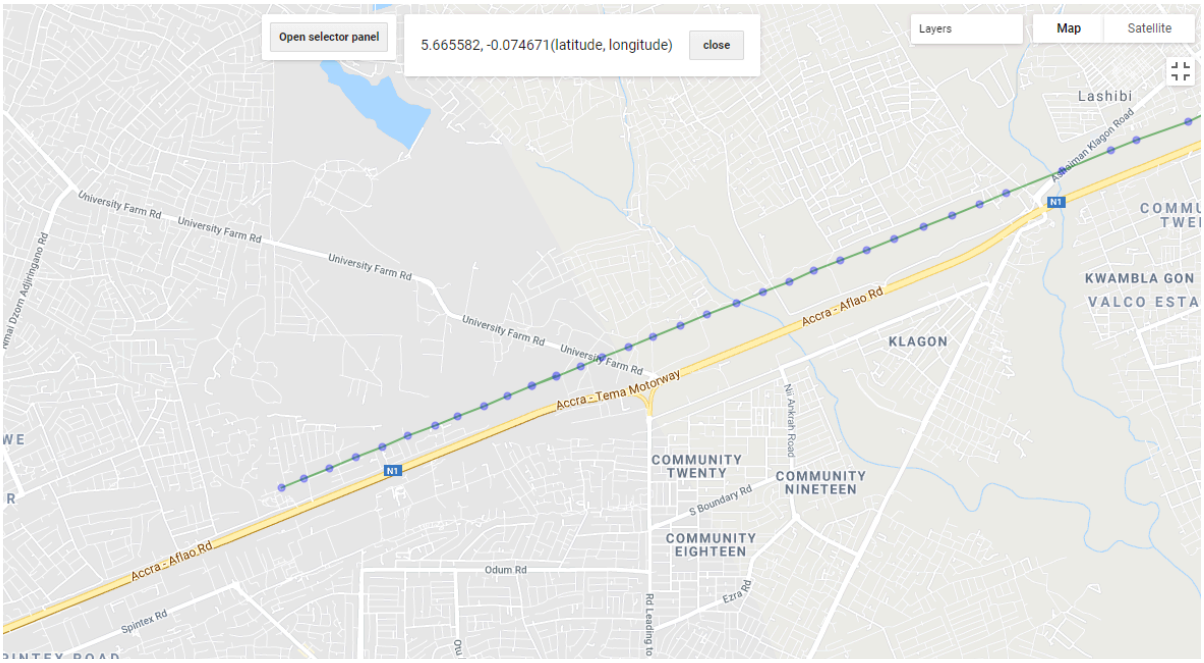


Figure 51 Towers superimposed on the map

Figure 52 towers superimposed on the map in satellite view

A navigation feature was included for navigating the corridor as a dropdown containing the tower names. The interface also has functionality for highlighting areas on the map using geometry drawing tools. Shown below is a screenshot of the map view and dropdown.

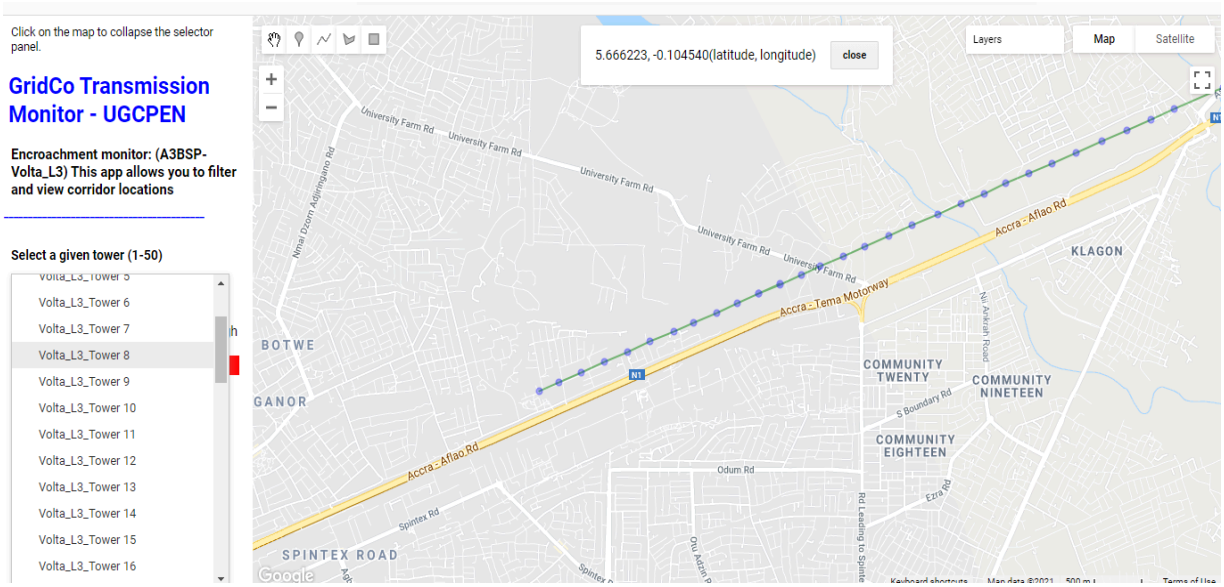


Figure 53 Map view with tower location selection

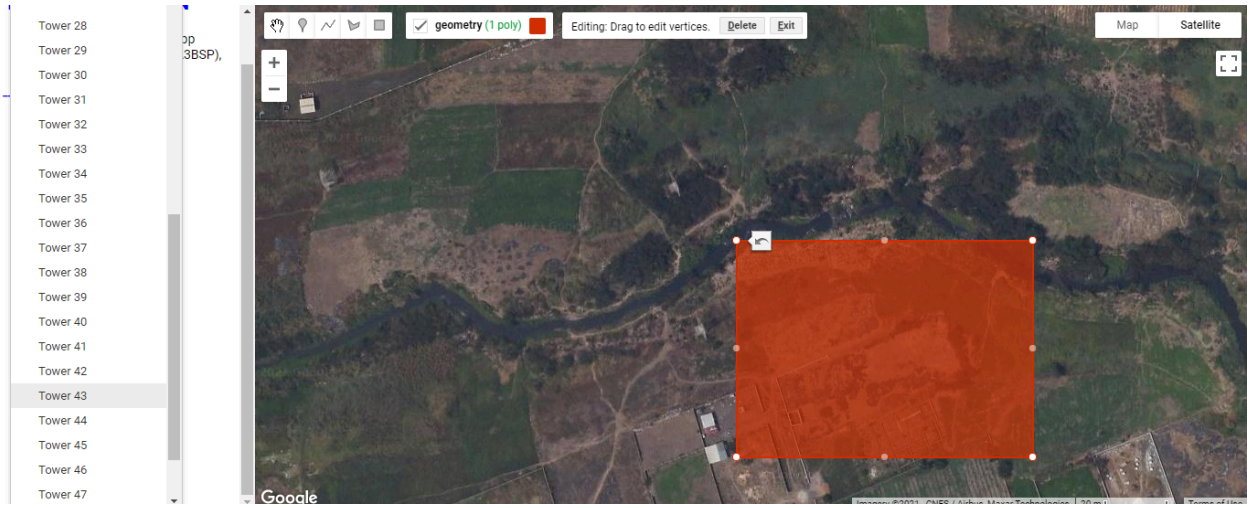


Figure 54 Map view showing a highlighted area of the map in satellite view.

The 50m transmission line buffer was created as a GEE geometry object and the pre-processed satellite image was clipped to the transmission line buffer for anomaly detection as shown in Figure 55.

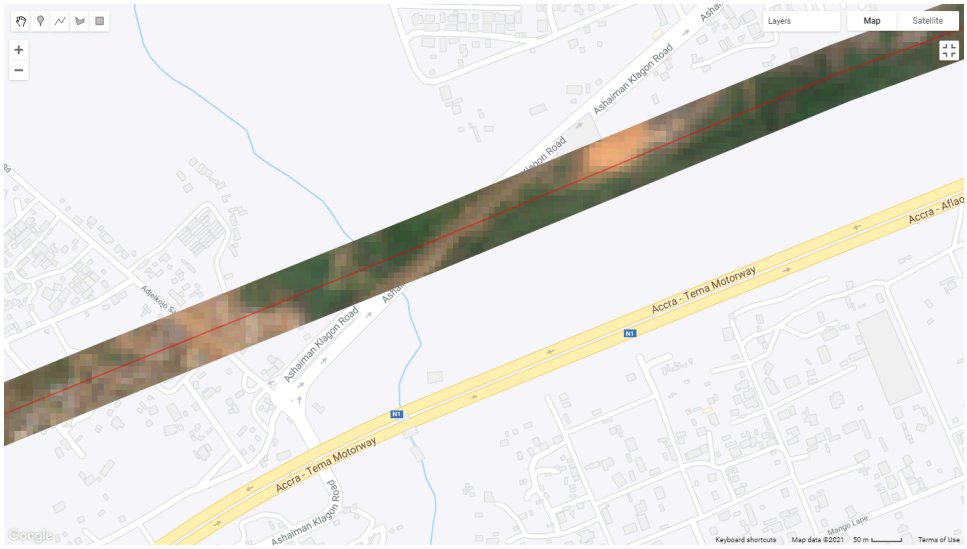


Figure 55 Transmission line buffer on satellite image

We highlight areas of change using a legend of colours ranging from cyan to red for areas of low to high change respectively. The colour value assigned to an area was dependent on the extent of change.

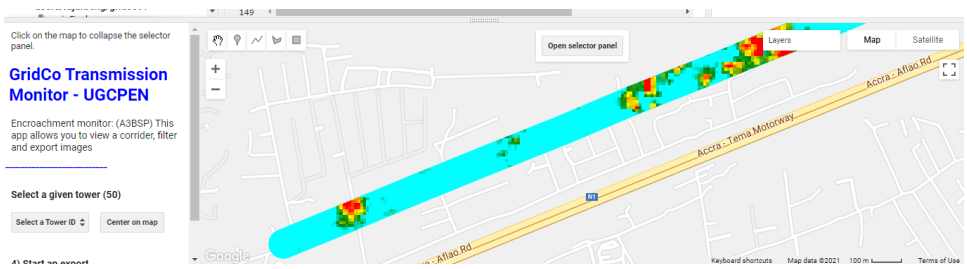


Figure 56 Change detection system prototype

Figure 57 Encroachment detection in corridor

The differenced images contained areas which had been recently constructed as well as areas which were undergoing construction in the test area. However, some of the detections shown were in areas that were false positives. To tackle this, we set a threshold value and took only changes whose values were greater than this threshold as shown in Figure 58.

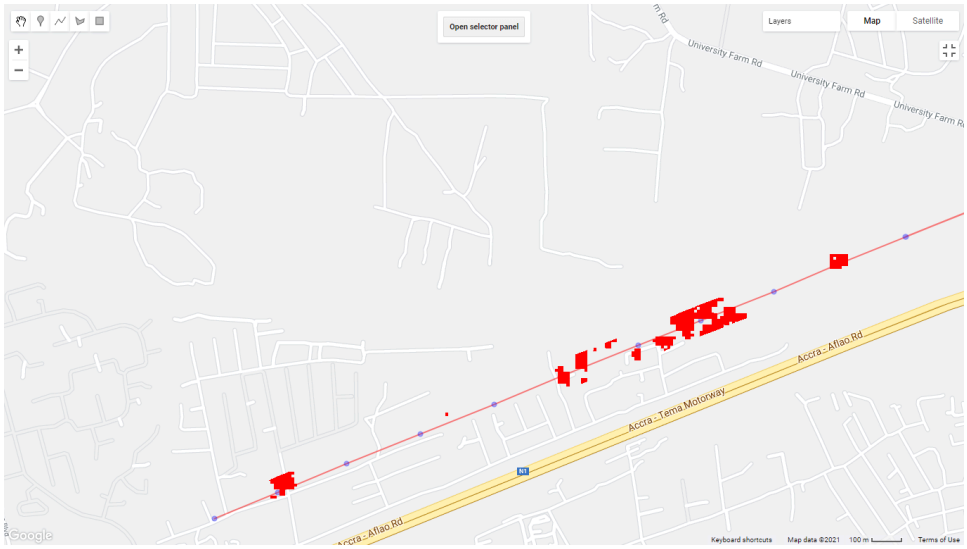


Figure 58 Change detection with thresholding

The model succeeded in classifying areas of construction around the test area but also classified areas of vegetation cover change. This resulted in a great number of high detections (shown in red below) because of changes in vegetation cover over the time period.

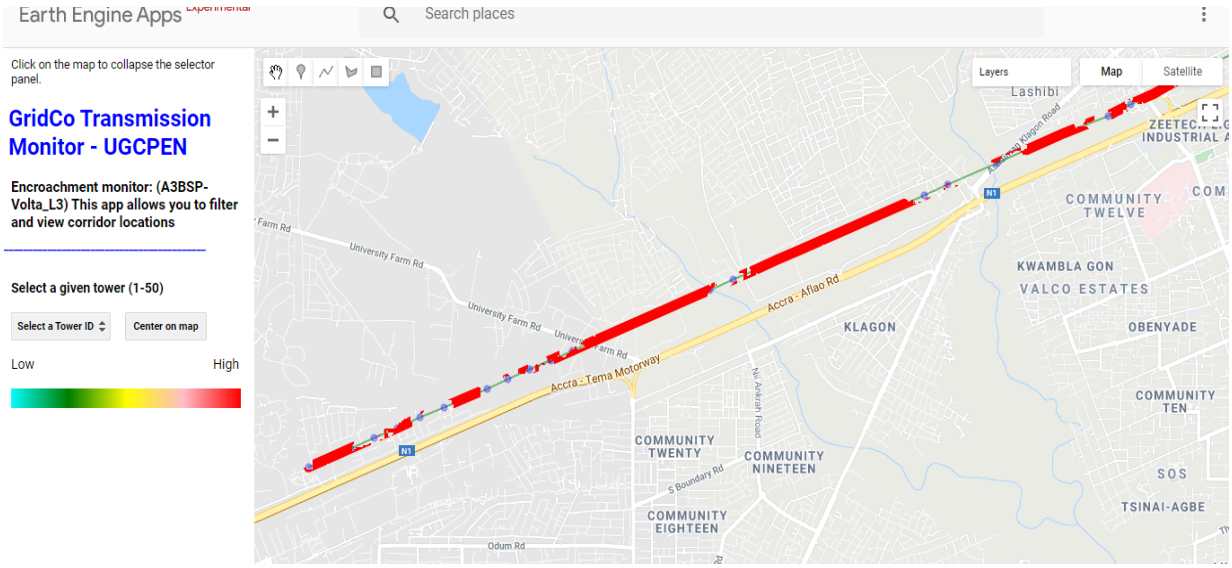


Figure 59 Change detection model in operation

To remove detections due to vegetation, change the areas with NDVI values above a certain value were extracted and applied as an inverse mask to eliminate them from the detection results. We added a map layer that highlights only areas above a certain threshold value in purple after filtering for changes due

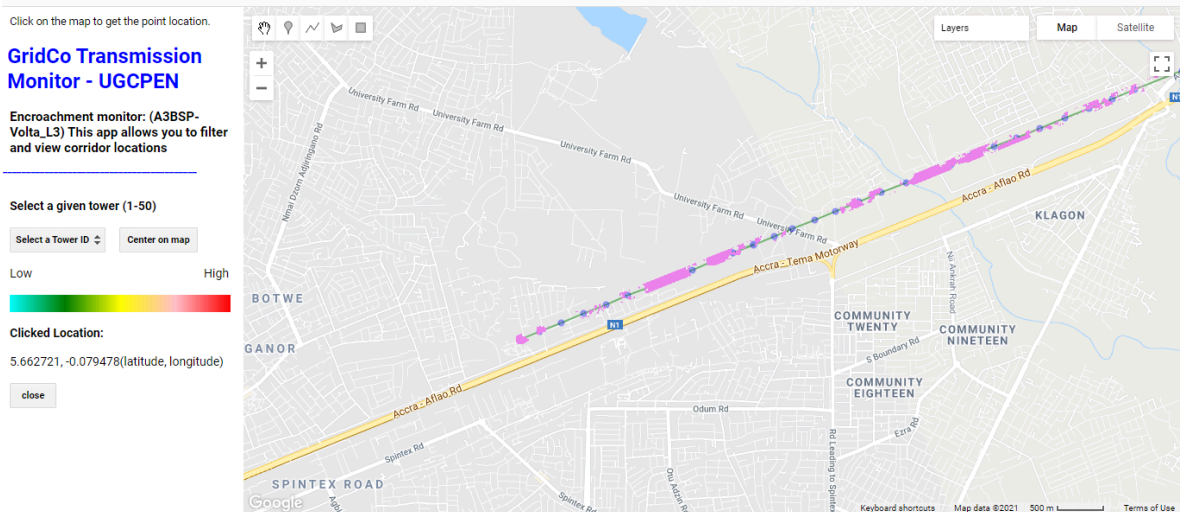
to vegetation using the NDVI. To reduce detections due to vegetation we calculate the Normalized Difference Vegetation Index (NDVI) which is a dimensionless index ranging from +1 to -1 for quantifying vegetation greenness and density[19]. Areas of bare land such as rock or sand show lower NDVI values of about 0.1 whilst moderate or dense vegetation cover has higher values. The NDVI is computed using the red and near-infrared bands of Sentinel-2 which correspond to band 4 (b4) and band 8 (b8) of the satellite image respectively.

$$NDVI = (NIR - R) / (NIR + R) \ldots\ldots\ldots\ldots (4.1)$$

NIR= Near Infrared band

R = Red band

The NDVI helps clearly distinguish areas of vegetation and built-up areas. We add the NDVI result as an image band to our image collection and mask the resultant image obtained from image differencing with the NDVI mask to obtain an image with areas of vegetation greater than an NDVI threshold masked out from the change detection output.



Some places which were classified as a high change were false positives detected as a result of changes in cloud cover which had not be eliminated by filtering and masking. To tackle this problem, we added a wider buffer which was different from the right-of-way for viewing the detections corridor so that the user can visually confirm detections due to clouds and ignore those areas when performing confirmation.

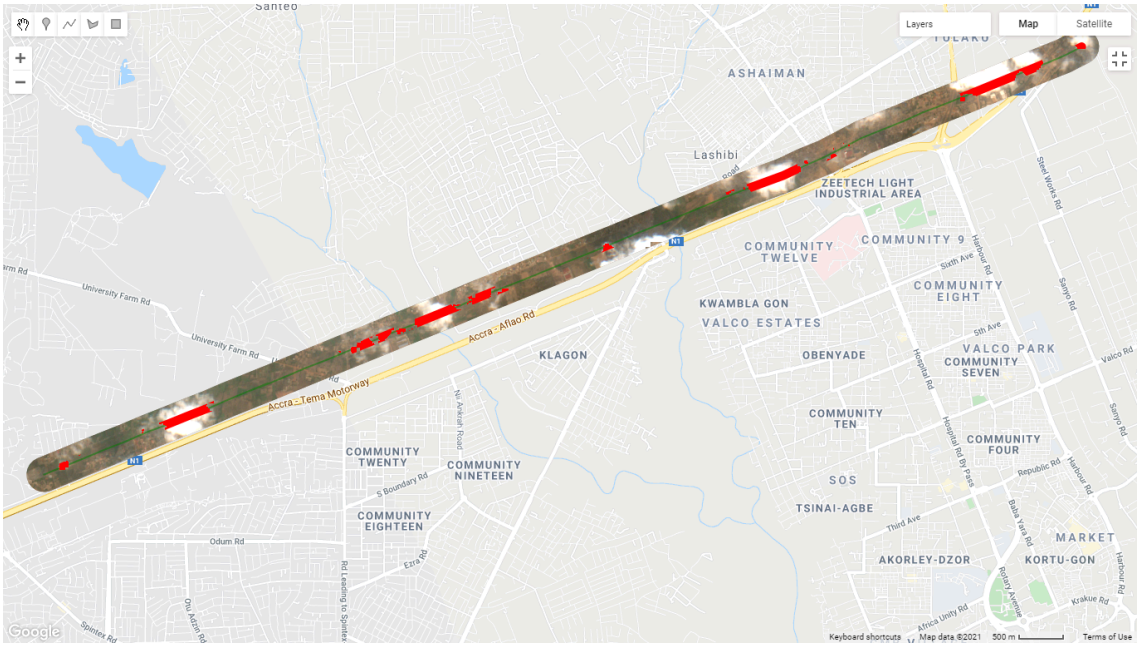


Figure 60 Clouds in the buffer

Furthermore, to eliminate false detections due to clouds we crosscheck results using Google Earth Pro's historical viewer (Figure 61), which allows us to view an area over given time periods. The images in the historical viewer are updated less frequently but are of higher resolution.

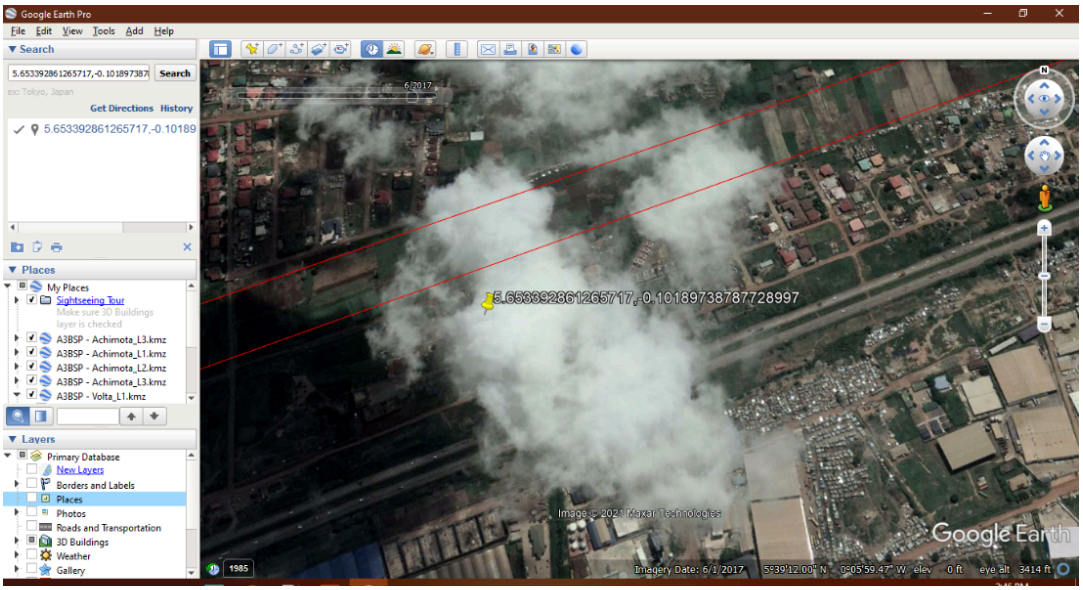


Figure 61 Detection confirmation in Google Earth Pro

4.3 Detection System Testing and Result

CNN Web Application Result and Analysis

After the tests in Chapter 4.3.1 were done, the system was able to detect the various structures within the corridor images. Though there were minor wrong classification of the system, due to the resolution and size of the image segmented. The table in Table 11 Image Classification and Accuracy shows each of the tower regions, its image classification and accuracy of it.

Table 11 Image Classification and Accuracy

Tower Images	Total segmented images	Correctly Classified images	Wrongly Classified images	Accuracy (%)
Tower2 to tower3	84	80	4	95.24
Tower3 to tower4	84	82	2	97.61
Tower4 to tower5	84	84	0	100
Tower12 to tower13	84	80	4	95.24
Tower13 to tower14	84	81	3	96.5

From the result obtained after submission, the model was able to classify and detect encroachment within the ROW (Right of Way) area of the transmission line with minor wrong classification based on the resolution of the image being too obscure. From Table 11 Image Classification and Accuracy, the model was able to perform image classification with an average score of 97%.

Change Detection System Result and Analysis

The system detected cases of encroachment over an area which was close to the Ashaiman market at 5.676775, -0.045459 (latitude and longitude respectively). Upon confirmation in Google Earth Pro, it was observed to be an accurate detection with the area showing high encroachment and a rapidly growing number of settlements within the corridor from 2015 to 2021.

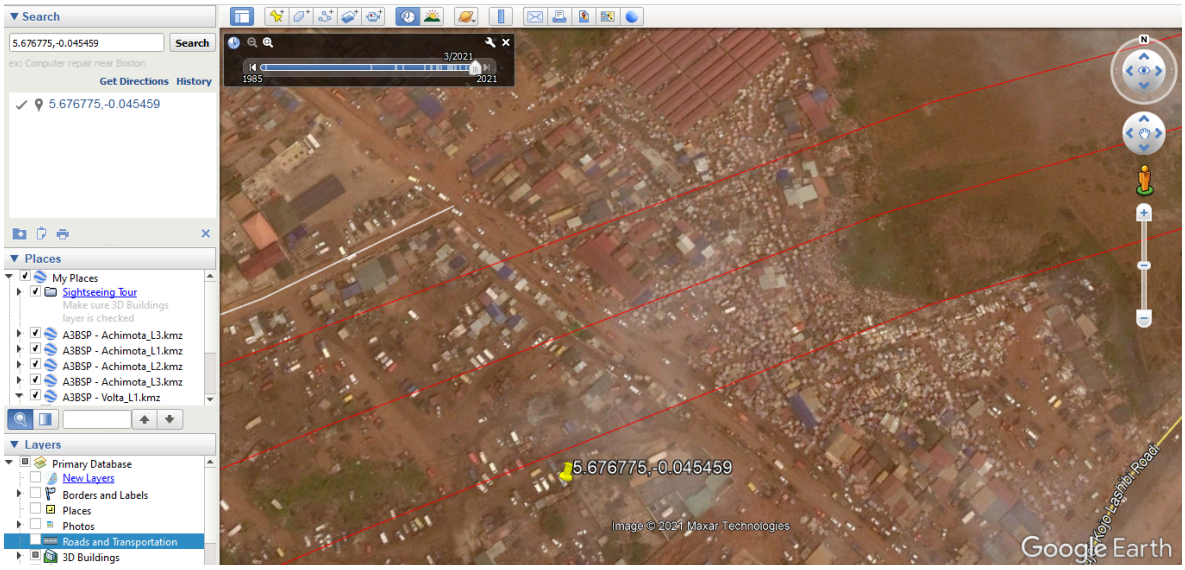


Figure 62 Encroached Area confirmation 2021

Figure 63 Encroached Area confirmation 2020

Other areas of encroachment that were also detected are given in Table 12.

Table 12 Detected areas of encroachment

Detection percentage %	Latitude	longitude
60	5.654973755	-0.099819517

90	5.665308633	-0.074241971
80	0.074241971	-0.048836088
90	5.682749796	-0.031498289
60	5.695254718	-0.000753133
60	5.675269130	- 0.050363271

Overall, the system was capable of detecting small settlements such as kiosks and large structures such as buildings as well as areas of constructions and was able to detect areas in the corridor which had been taken over by settlements. By performing thresholding on the results, the detection accuracy increased by 30%. Vegetation masking using NDVI values helped eliminate false detections due to vegetation.

CHAPTER 5 - CONCLUSION AND RECOMMENDATION

5.1 Introduction

This chapter contains a summary of the work done and concludes on the implementation of the detection system and possible recommendations for the improvement of the encroachment detection system.

5.2 Conclusions

The implementation of this project has provided an alternative approach to transmission corridor monitoring by using satellite imagery and a CNN model to overcome the shortcomings of conventional monitoring methods. We trained a CNN model to detect encroachment using a labelled dataset and built an application that uses the model to detect encroachment in a transmission corridor as well as an alert system to alert the utilities. The CNN model reached an accuracy of 97.50% and proved suitable for anomaly detection. We were able to:

- Build a web-application for real time monitoring using satellite images and a trained CNN model to detect encroachment in a chosen corridor.
- Provide a system to provide an alert to the utility of any potential obstruction in the right-of-way corridor for quick intervention.
- Create functionality for tracking an area view changes in the transmission corridor.

The application would be of benefit in reducing the time and cost involved in conventional monitoring using vehicles, facilitate line maintenance and provide real-time information for quick intervention of possible encroachment. The completion of this project has provided a suitable and effective means to detect encroachment in a transmission corridor.

5.3 Recommendations

The CNN model can be trained with a greater number of datasets to allow for greater accuracy in detection by allowing the model to generalize based on the input data. We were unable to add detection with ‘YOLO’ as stated in Chapter 2 for faster processing and detections. The system would benefit from a frequently updated source of high resolutions satellite images aside from Google Maps built into the application to correctly view the transmission corridor. The system can be implemented over more than one corridor instead of the chosen pilot corridor to provide timely monitoring of the utilities transmission corridors for encroachment prevention. The use of commercial satellite providers such as Planet [20] a commercial satellite service that allows users to specify an area of interest for monitoring using their constellation of satellites and Spymesat [21] a mobile application that notifies a user when imaging satellites are flying over an area could also be incorporated into the work.

5.4 Observations and Challenges

The acquisition of datasets of structures such as buildings, kiosks, uncompleted structures, etc proved a challenge because the structures to be detected differed from the datasets readily available. We had to perform our own dataset acquisition by cropping satellite images to train and validate the CNN model. The resolution of the 10m resolution Sentinel-2 image collection was not suitable for viewing the right-of-way corridor of the transmission corridor at a high zoom level for visual inspection. The system

was reliant on an external application, Google Earth Pro whose imagery update frequency is random, to confirm possible encroachment. Cloud cover also presented a challenge in obtaining images with a clear view of the transmission corridor when using recently updated satellite images.

References

[1] "Ghana: Accra: Areas Experiencing Power Outage Will Be Restored By 5PM-GRIDCo | Energy News Africa." https://energynewsafrica.com/index.php/2020/05/11/ghana-accra-areas-experiencing-power-outage-will-be-restored-by-5pm-gridco/ (accessed Sep. 08, 2021).

[2] A. Zain, "Transmission Lines Monitoring from Satellite Images," *J. Mech. Contin. Math. Sci.*, vol. 13, no. 5, 2018, doi: 10.26782/jmcms.2018.10.00010.

[3] "Is living near power lines bad for our health? | British Columbia Medical Journal." https://bcmj.org/bccdc/living-near-power-lines-bad-our-health (accessed Aug. 29, 2021).

[4] F. M. E. Haroun, S. N. M. Deros, and N. M. Din, "A review of vegetation encroachment detection in power transmission lines using optical sensing satellite imagery," *Int. J. Adv. Trends Comput. Sci. Eng.*, vol. 9, no. 1.4 Special Issue, pp. 618–624, 2020, doi: 10.30534/ijatcse/2020/8691.42020.

[5] V. Ostankovich and I. Afanasyev, "Illegal Buildings Detection from Satellite Images using GoogLeNet and Cadastral Map," *9th Int. Conf. Intell. Syst. 2018 Theory, Res. Innov. Appl. IS 2018 - Proc.*, pp. 616–623, 2018, doi: 10.1109/IS.2018.8710565.

[6] S. Ji, Y. Shen, M. Lu, and Y. Zhang, "Building instance change detection from large-scale aerial images using convolutional neural networks and simulated samples," *Remote Sens.*, vol. 11, no. 11, p. 1343, Jun. 2019, doi: 10.3390/rs11111343.

[7] N. Celik, "Change Detection of Urban Areas in Ankara through Google Earth Engine," *2018 41st Int. Conf. Telecommun. Signal Process. TSP 2018*, no. July 2018, pp. 1–5, Aug. 2018, doi: 10.1109/TSP.2018.8441377.

[8] A. Lefebvre, C. Sannier, and T. Corpetti, "Monitoring Urban Areas with Sentinel-2A Data : Application to the Update of the Copernicus High Resolution Layer Imperviousness Degree," pp. 1–21, 2016, doi: 10.3390/rs8070606.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, vol. 2016-December, pp. 779–788, Dec. 2016, doi: 10.1109/CVPR.2016.91.

[10] M. Radovic, O. Adarkwa, and Q. Wang, "Object recognition in aerial images using convolutional neural networks," *J. Imaging*, vol. 3, no. 2, 2017, doi: 10.3390/jimaging3020021.

[11] J. Haas and Y. Ban, "Sentinel-1A SAR and sentinel-2A MSI data fusion for urban ecosystem service mapping," *Remote Sens. Appl. Soc. Environ.*, vol. 8, pp. 41–53, Nov. 2017, doi: 10.1016/J.RSASE.2017.07.006.

[12] R. Goldblatt, K. Deininger, and G. Hanson, "Utilizing publicly available satellite data for urban research: Mapping built-up land cover and land use in Ho Chi Minh City, Vietnam," *Dev. Eng.*, vol. 3, pp. 83–99, Jan. 2018, doi: 10.1016/J.DEVENG.2018.03.001.

[13] J. Brownlee, "A Gentle Introduction to the Rectified Linear Unit (ReLU)," Sep. 09, 2019. https://machinelearningmastery.com/rectified-linear-activation-function-for-deep-learning-neural-networks/ (accessed Sep. 24, 2021).

[14] J. Brownlee, “Softmax Activation Function with Python.” https://machinelearningmastery.com/softmax-activation-function-with-python/ (accessed Sep. 24, 2021).

[15] S. Verma, “Understanding different Loss Functions for Neural Networks.” https://towardsdatascience.com/understanding-different-loss-functions-for-neural-networks-dd1ed0274718 (accessed Sep. 24, 2021).

[16] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau, and R. Moore, “Google Earth Engine: Planetary-scale geospatial analysis for everyone,” *Remote Sens. Environ.*, vol. 202, pp. 18–27, 2017, doi: 10.1016/j.rse.2017.06.031.

[17] “Earth Engine Code Editor | Google Earth Engine | Google Developers.” https://developers.google.com/earth-engine/guides/playground (accessed Sep. 04, 2021).

[18] “UTM to Latitude and Longitude Converter.” https://www.engineeringtoolbox.com/utm-latitude-longitude-d_1370.html (accessed Aug. 28, 2021).

[19] J. Brown, “NDVI, the Foundation for Remote Sensing Phenology.” https://www.usgs.gov/core-science-systems/eros/phenology/science/ndvi-foundation-remote-sensing-phenology?qt-science_center_objects=0#qt-science_center_objects (accessed Sep. 15, 2021).

[20] “Planet | Homepage.” https://www.planet.com/ (accessed Aug. 22, 2021).

[21] “Mobile App SpyMeSat.” https://www.spymesat.com/tour-the-app.html (accessed Aug. 24, 2021).

Appendices

Appendix A1 Convolutional Neural Network model codes

```
import cv2
import os
import matplotlib.pyplot as plt
from glob import glob
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dropout, BatchNormalization
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing import image
import numpy as np
import tensorflow as tf
 #Setting the training and testing data directory
train_datagenerator = ImageDataGenerator(rescale = 1.0/255,
                                         shear_range = 0.2,
                                         zoom_range = 0.5,
                                         horizontal_flip = True,
                                         rotation_range=10,
                                         width_shift_range=0.2,
                                         brightness_range=[0.2,1.2]
                                         )
test_datagenerator = ImageDataGenerator(rescale = 1.0/255)


train_data = train_datagenerator.flow_from_directory(r'D:\Mystuff\New folder\PROJECT
L400\project\code\version 2\basedata\train',
                                                     target_size = (128,128),
                                                     batch_size = 30,
                                                     class_mode = 'categorical',
```

```
                                        shuffle=True,)
test_data = test_datagenerator.flow_from_directory(r'D:\Mystuff\New folder\PROJECT
L400\project\code\version 2\basedata\test',
                                                   target_size = (128,128),
                                                   batch_size = 30,
                                                   class_mode = 'categorical',
                                                   shuffle=False,)


#Creating and training the model
cnn = Sequential()
# Convolution
cnn.add(Conv2D(filters=16,padding =
"same",kernel_size=3,activation='relu',input_shape=[128,128,3]))
# pooling
cnn.add(MaxPooling2D(pool_size=3,strides=3))
cnn.add(Dropout(0.4))
# Convolution
cnn.add(Conv2D(filters=32,padding = "same",kernel_size=3,activation='relu'))
#pooling
cnn.add(MaxPooling2D(pool_size=3,strides=3))
cnn.add(Dropout(0.4))
cnn.add(Conv2D(filters=64,padding = "same",kernel_size=3,activation='relu'))
#pooling
cnn.add(MaxPooling2D(pool_size=3,strides=3))
cnn.add(Dropout(0.4))
#flattening
cnn.add(Flatten())
cnn.add(Dense(units=128,activation='relu'))
cnn.add(Dense(units=64,activation='relu'))
# Output layer
```

```
cnn.add(Dense(units=2,activation='softmax'))
cnn.compile(optimizer=
tf.keras.optimizers.Adam(learning_rate=0.001),loss='categorical_crossentropy',metrics=['accuracy'])
history = cnn.fit(x = train_data,
                  validation_data = test_data,
                  epochs = 25,
                    callbacks=[
                      tf.keras.callbacks.EarlyStopping(
                          monitor='accuracy',
                          patience=3,
                          restore_best_weights=True
                      )
                    ]
                 )
#Evaluating the model
score = cnn.evaluate(test_data)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
# model.save("test_model.h5")
```

Appendix A2 Image Segmentation and Unification codes

```
#Segmenting the Images

def crop_one_picture(path, filename, cols, rows):

    ##Read the color image, the transparency of the image (alpha channel) is ignored, the default parameter;

    ##grayscale image; read the original image, including the alpha channel; can be expressed as 1, 0, -1

    full_path = os.path.join(path,filename)

    img = cv2.imread(full_path,-1)

#     img = cv2.imread(r'D:\Mystuff\New folder\PROJECT L400\project\code\version 2\crop\tower3_to_twer4.jpg',-1)

    sum_rows = img.shape[0]  # height

    sum_cols = img.shape[1]  # width

    save_path = path + "\\crop_images\\".format(cols, rows)  # Saved path

    if not os.path.exists(save_path):

        os.makedirs(save_path)

    print("Cropped {0} column pictures, {1} row pictures.".format(int(sum_cols / cols), int(sum_rows / rows)))

    for i in range(int(sum_cols / cols)):

        for j in range(int(sum_rows / rows)):

            cv2.imwrite(

                save_path + os.path.splitext(filename)[0] + '_' + str(j) + '_' + str(i) + os.path.splitext(filename)[1],

                img[j * rows:(j + 1) * rows, i * cols:(i + 1) * cols, :])

            #
print(path+"\crop\\"+os.path.splitext(filename)[0]+'_'+str(j)+'_'+str(i)+os.path.splitext(filename)[1])

    print("Cropping completed, get {0} pictures.".format(int(sum_cols / cols) * int(sum_rows / rows)))

    print("File saved in {0}".format(save_path))

    """Traversing a certain format picture under the folder """

path = r'D:\Mystuff\New folder\PROJECT L400\project\code\version 2\pictures\crop'  # Folder of the picture to be cropped

filename = 'tower2_to_twer3.jpg'  # Image name to be cropped
```

```
cols = 200  # Width of small pictures (number of columns)
rows = 200  # Small picture height (number of lines)
crop_one_picture(path, filename, 100, 100)
#Code for Prediction and Drawing of Contour Boxes
from tensorflow.keras.preprocessing import image

def Model_Predictions(directory):
    for filename in os.listdir(directory):
        pred = ["House","Land"]
        input_path = os.path.join(directory, filename)
        test_image = image.load_img(input_path, target_size = (128,128,3))
        land_image = image.load_img(input_path, target_size = (128,128,3))
        plt.imshow(test_image)
        plt.show()
        test_image = image.img_to_array(test_image)
        test_image = np.expand_dims(test_image,axis=0)
        result = cnn.predict(test_image)
        result_value =pred[result.argmax()]
        if result_value == 'House':
            print('This is a house with stuff')
    ## show contours code
            img = cv2.pyrDown(cv2.imread(input_path, cv2.IMREAD_UNCHANGED))

            # threshold image
            ret, threshed_img = cv2.threshold(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY),
                        127, 255, cv2.THRESH_BINARY)
            # find contours and get the external one
            contours, hier = cv2.findContours(threshed_img, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
            for c in contours:
```

```
            # get the bounding rect
            x, y, w, h = cv2.boundingRect(c)
            # draw a green rectangle to visualize the bounding rect
            cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 1)
            # get the min area rect
            rect = cv2.minAreaRect(c)
            box = cv2.boxPoints(rect)
            # convert all coordinates floating point values to int
            box = np.int0(box)
        print(len(contours))
        # cv2.drawContours(img, contours, -1, (255, 255, 0), 1)
        directory_img = r'D:\Mystuff\New folder\PROJECT L400\project\code\version 2\pictures\image_test'
        os.chdir(directory_img)
        # cv2.imshow("contours", img)
        cv2.imwrite(filename,img)
        # cv2.imshow("contours", img)
   ## end of contour code
      elif result_value == 'Land':
        print('This is a Bare Land')
        directory_img = r'D:\Mystuff\New folder\PROJECT L400\project\code\version 2\pictures\image_test'
        os.chdir(directory_img)
        img = cv2.pyrDown(cv2.imread(input_path, cv2.IMREAD_UNCHANGED))
        # cv2.imshow("contours", img)
        cv2.imwrite(filename,img)
directory = r'D:\Mystuff\New folder\PROJECT L400\project\code\version 2\pictures\crop\crop_images'

Model_Predictions(directory)
# Merging of Pictures
```

```
def file_name(root_path, picturetype):
    filename = []
    for root, dirs, files in os.walk(root_path):
        for file in files:
            if os.path.splitext(file)[1] == picturetype:
                filename.append(os.path.join(root, file))
    return filename
def merge_picture(merge_path,num_of_cols,num_of_rows):
    filename=file_name(merge_path,".jpg")
    shape=cv2.imread(filename[0]).shape    #Three-channel images need to change -1 to 1
    cols=shape[1]
    rows=shape[0]
    channels=shape[2]
    dst=np.zeros((rows*num_of_rows,cols*num_of_cols,channels),np.uint8)
    for i in range(len(filename)):
        img=cv2.imread(filename[i],-1)
        cols_th=int(filename[i].split("_")[-1].split('.')[0])
        rows_th=int(filename[i].split("_")[-2])
        roi=img[0:rows,0:cols,:]
        dst[rows_th*rows:(rows_th+1)*rows,cols_th*cols:(cols_th+1)*cols,:]=roi
    #merge destination
#     cv2.imwrite(merge_path+"merge.jpg",dst)
    directory_merge = r'D:\Mystuff\New folder\PROJECT L400\project\code\version 2\pictures\merge'
    os.chdir(directory_merge)
    cv2.imwrite("asdfgfs.jpg",dst)
    print('File merged and saved in {}'.format(directory_merge))
merge_path = r"D:\Mystuff\New folder\PROJECT L400\project\code\version 2\pictures\image_test"  # The folder containing the small pictures to be merged
num_of_cols = 12  # Number of columns
num_of_rows = 7 # Rows
```

```
merge_picture(merge_path, num_of_cols, num_of_rows)
```

Appendix A3 Google Earth Engine codes

```
//////////////////////TOWERS
var towerCoordinates =[
  [ -0.10734454474815136,5.651902335725393],
 [ -0.10540209288720703,5.65268890668925],
 [ -0.10330159488974688,5.653554187426227],
 [ -0.10105063067919451,5.654460989088002],
 [ -0.09880363358634225,5.655349276521264],
 [ -0.09666263848621792,5.656249195532561],
 [ -0.09439580148646032,5.657135823550668],
 [ -0.09250269067218042,5.6578995415120685],
 [ -0.09026979748346742,5.658760477219362],
 [ -0.08831820434040605,5.659603118389631],
 [ -0.08623078284268049,5.660427719983046],
 [ -0.08416921252515497,5.661258404122756],
 [ -0.08623898083714998,5.660429568189462],
 [ -0.08417945127433324,5.6612489690478025],
 [ -0.0821079032214036,5.662090434413582],
 [ -0.08028406488707107,5.662831767318834],
 [ -0.07805827573348934,5.663729353947401],
 [ -0.07604556533881368,5.664544432720229],
 [ -0.07366064104956704,5.665502628310298],
 [ -0.07142261472787048,5.666410506899543],
 [ -0.06894595789888891,5.66740498226597],
```

[-0.06676138452895586,5.668281574770141],
[-0.06454064105477242,5.669176222162455],
[-0.062433196625060194,5.670066012090303],
[-0.06022632727645982,5.670926735978312],
[-0.05795757114856759,5.671843599747985],
[-0.05566182337126113,5.672764740376744],
[-0.053146598074921 5,5.673783316869138],
[-0.05076497567072824,5.674741395395289],
[-0.04846579395308659,5.675661755181294],
[-0.046213636642455924,5.676574392010335],
[-0.04148613286914138,5.6784628107280275],
[-0.037405003818780536,5.680138477104199],
[-0.03520648478642974,5.681034975631185],
[-0.030896643565964776,5.682600446616676],
[-0.028707534899851783,5.68362985490974],
[-0.026529655555508125,5.684508976241601],
[-0.022733303106548863,5.686482289167362],
[-0.020555048494120154,5.687362188666672],
[-0.018415083916601738,5.688214268021817],
[-0.016497078934835763,5.68902972145079],
[-0.014044068385361097,5.6899858589819665],
[-0.011439840333703266,5.691023048152545],
[-0.009268313463771438,5.6919068968756825],
[-0.007267153953125049 5,5.692720158105726],
[-0.0056221306953117,5.693379601086451],
[-0.0035976610681132115,5.694199950335834],
[-0.0035997320639333807,5.694200593505222],
[-0.0019291842872353193,5.694859527315408],
[-0.00005742076352310832,5.6958145402924325],

```
 ]
//Map parameters
var zoomLevel = 14 ;
var cloudP = 15;
Map.style().set('cursor', 'crosshair');
 var vizParams = {
  bands:['B4','B3','B2'],min:0.02,max:0.3,
                    gamma:1.5
};
///////////////LOAD SENTINEL DATA
var s2=ee.ImageCollection("COPERNICUS/S2_SR")
 var vizParams = {
  bands:['B4','B3','B2'],min:0.02,max:0.3,
                    gamma:1.5
};
var towerLine;
function drawTowers(){
  var lnstring = ee.Geometry.LineString(towerCoordinates)
  towerLine =lnstring
  //set right-of-way buffer
  var lineStringBuffer = lnstring.buffer({'distance': 50});
  var lineDistanceBuffer = lnstring.buffer({'distance': 200});
var points = ee.Geometry.MultiPoint(towerCoordinates)
// Function to mask clouds S2
function maskS2srClouds(image) {
  var qa = image.select('QA60');
  // Bits 10 and 11 are clouds and cirrus, respectively.
  var cloudBitMask = 1 << 10;
  var cirrusBitMask = 1 << 11;
```

```
  // Both flags should be set to zero, indicating clear conditions.
  var mask = qa.bitwiseAnd(cloudBitMask).eq(0)
      .and(qa.bitwiseAnd(cirrusBitMask).eq(0));
  return image.updateMask(mask).divide(10000).select("B[1-9]");
}
var mask;
var startDate = '2021-01-01';//default start date
var endDate = ee.Date(Date.now());
var startDate_old= new Date('2020-01-01')
var endDate_old= new Date('2020-12-31')
var old_image= s2.filterBounds(lineStringBuffer )
              .filterDate(startDate_old,endDate_old)
              .filterMetadata('CLOUDY_PIXEL_PERCENTAGE',
                        'less_than',cloudP)
                        .map(maskS2srClouds)
                        // print(old_image, 'older')
              old_image=old_image
              .median()
              .clip(lineDistanceBuffer); // can be changed to mean, min, etc
var s2Filt = s2.filterBounds(lineStringBuffer )
              .filterDate(startDate,endDate)
              .filterMetadata('CLOUDY_PIXEL_PERCENTAGE',
                        'less_than',cloudP)
              .sort('CLOUDY_PIXEL_PERCENTAGE')
              .map(maskS2srClouds);
var s2composite = s2Filt.median().clip(lineDistanceBuffer); // can be changed to mean, min, etc
Map.addLayer(old_image,vizParams,'old image S2', !true);
Map.addLayer(s2composite,vizParams,'Recent image S2 ', !true);
var new_image = s2composite;
```

```
//Add towers and lines to the map
Map.addLayer(lnstring, {color: 'green'}, 'Transmission Line', true, 0.5)
Map.addLayer(points, {color: 'blue', }, 'Tower points', true, 0.4)
//////////////////CHANGE DETECTOR
var magnitude = function(image) {
   return image
   .abs()
   .reduce('sum')
};
var difference = magnitude(
  new_image.subtract(old_image));
Map.addLayer(difference.clip(lineStringBuffer), {min:0,max:0.6 ,palette: ['cyan', 'green', 'yellow','pink', 'red']}, 'Imagery difference', !true);
  var threshold=0.2;
  var differencethreshold = difference.gte(threshold); // edit threshold value here
var maskdifferencethreshold = differencethreshold.updateMask(differencethreshold);
Map.addLayer(maskdifferencethreshold.clip(lineStringBuffer), {min:threshold,max:0.6, palette: ['blue', 'orange', 'red']}, 'Track Changes in Corridor',!true);
function addNdvi(input) {
  var ndvi = input.normalizedDifference(['B8', 'B4']);
  return input.addBands(ndvi);
}
  var new_veg = s2Filt.map(addNdvi)
  .select(['nd']).median().clip(lineStringBuffer);
  var differencethreshold_ndvisd = new_veg.lte(0.25);
var maskdifferencethreshold_ndvisd = differencethreshold_ndvisd.mask(differencethreshold_ndvisd);
var maskdifferencethreshold_ndvisd_mask = maskdifferencethreshold_ndvisd.add(maskdifferencethreshold);
Map.addLayer(maskdifferencethreshold_ndvisd_mask, {palette: ['white', 'green', 'violet']}, 'Track Changes (no agriculture)',true);
  }
```

```
var i =0, counter =1, selectedTower ='Tower 1';
var APPSTRINGS= {
  title: 'GridCo Transmission Monitor - UGCPEN',
  subtitle: 'Encroachment monitor: (A3BSP-Volta_L3) This app allows you to filter and view corridor locations'
}
var towers = []
function initializeTowerArray(){
  //returns a tower i with its location
  var changeTowerNames =function (index){
    return {
      location:  towerCoordinates[index],
      name: 'Volta_L3_Tower ' + (index+1)
    }
  }
//initialize a counter for tower names
    for(i=0; i<towerCoordinates.length; i++){
      towers.push(changeTowerNames(i))
    }
}
var findTower = function(element){
  return element.name === selectedTower;
}
initializeTowerArray();
// Get tower names
var towerNames = towers.map(function(element){
    return element.name}
  )
drawTowers()
var app = {};
```

```
/** Creates the UI panels. */
app.createPanels = function() {
  /* The introduction section. */
  app.intro = {
    panel: ui.Panel([
      ui.Label({
        value: APPSTRINGS.title,
        style: {fontWeight: 'bold', fontSize: '24px', margin: '10px 5px', color: 'blue'}
      }),
      ui.Label(
      {
        value: APPSTRINGS.subtitle,
        style: {fontWeight: 'bold', fontSize: '15px'
        // , margin: '10px 5px', color: 'blue'
        }
      }
              ),
              ui.Label(
              {
                value: '_____',
      style: {fontWeight: 'bold', fontSize: '12px', margin: '0',padding: '0', color: 'blue'}
              })
    ])
  };
  /* The collection filter controls. */
  app.filters = {
    mapCenter: ui.Checkbox({label: 'Filter to map center', value: true}),
    startDate: ui.Textbox('YYYY-MM-DD', '2020-05-01'),
```

```
  endDate: ui.Textbox('YYYY-MM-DD', '2021-01-01'),
  applyButton: ui.Button('Apply filters', app.applyFilters),
  loadingLabel: ui.Label({
    value: 'Loading...',
    style: {stretch: 'vertical', color: 'gray', shown: false}
  })
};
app.filters.panel = ui.Panel({
  widgets: [
    ui.Label('1) Select dates', {fontWeight: 'bold'}),
    ui.Label('Start date', app.HELPER_TEXT_STYLE), app.filters.startDate,
    ui.Label('End date', app.HELPER_TEXT_STYLE), app.filters.endDate,
    app.filters.mapCenter,
    ui.Panel([
      app.filters.applyButton,
      app.filters.loadingLabel
    ], ui.Panel.Layout.flow('horizontal'))
  ],
  style: app.SECTION_STYLE
});
app.picker = {
  select: ui.Select({
    placeholder: 'Select a Tower ID',
    onChange: app.refreshMapLayer
  }),
  centerButton: ui.Button('Center on map', function() {
    Map.centerObject(towerLine, zoomLevel-1);
  })
};
```

```
app.picker.panel = ui.Panel({
  widgets: [
    ui.Label('Select a given tower (1-'+towers.length+')', {fontWeight: 'bold'}),
    ui.Panel([
      app.picker.select,
      app.picker.centerButton
    ], ui.Panel.Layout.flow('horizontal'))
  ],
  style: app.SECTION_STYLE
});
/* The visualization section. */
app.vis = {
  label: ui.Label(),
  // Create a select with a function that reacts to the "change" event.
  select: ui.Select({
    items: Object.keys(app.VIS_OPTIONS),
    onChange: function() {
      // Update the label's value with the select's description.
      var option = app.VIS_OPTIONS[app.vis.select.getValue()];
      app.vis.label.setValue(option.description);
      app.refreshMapLayer();
    }
  })
};
/* The panel for the visualization section with corresponding widgets. */
app.vis.panel = ui.Panel({
  widgets: [
    ui.Label(' Select Visualization', {fontWeight: 'bold'}),
    app.vis.select,
```

```
    app.vis.label
  ],
  style: app.SECTION_STYLE
});
app.vis.select.setValue(app.vis.select.items().get(0));
/* The export section. */
app.export = {
  button: ui.Button({
    label: 'Export the current image to Drive',
    // React to the button's click event.
    onClick: function() {
      // Select the full image id.
      var imageIdTrailer = app.picker.select.getValue();
      var imageId = app.COLLECTION_ID + '/' + imageIdTrailer;
      // Get the visualization options.
      var visOption = app.VIS_OPTIONS[app.vis.select.getValue()];
      // Export the image to Drive.
      Export.image.toDrive({
        image: ee.Image(imageId).select(visOption.visParams.bands),
        description: 'L8_Export-' + imageIdTrailer,
      });
    }
  })
};
/* The panel for the export section with corresponding widgets. */
app.export.panel = ui.Panel({
  widgets: [
    ui.Label('4) Start an export', {fontWeight: 'bold'}),
    app.export.button
```

```
    ],
    style: app.SECTION_STYLE
  });
};
app.createHelpers = function() {
  app.setLoadingMode = function(enabled) {
    // Set the loading label visibility to the enabled mode.
    app.filters.loadingLabel.style().set('shown', enabled);
    var loadDependentWidgets = [
      app.vis.select,
      app.filters.startDate,
      app.filters.endDate,
      app.filters.applyButton,
      app.filters.mapCenter,
      app.picker.select,
      app.picker.centerButton,
      app.export.button
    ];
    loadDependentWidgets.forEach(function(widget) {
      widget.setDisabled(enabled);
    });
  };
  /** Applies the selection filters currently selected in the UI. */
  app.applyFilters = function() {
    app.setLoadingMode(true);
    var filtered = ee.ImageCollection(app.COLLECTION_ID);
    if (app.filters.mapCenter.getValue()) {
      filtered = filtered.filterBounds(Map.getCenter());
    }
```

```
    var start = app.filters.startDate.getValue();
    if (start) start = ee.Date(start);
    var end = app.filters.endDate.getValue();
    if (end) end = ee.Date(end);
    if (start) filtered = filtered.filterDate(start, end);
    var computedIds = filtered
        .limit(app.IMAGE_COUNT_LIMIT)
        .reduceColumns(ee.Reducer.toList(), ['system:index'])
        .get('list');
    computedIds.evaluate(function(ids) {
      app.setLoadingMode(false);
      app.picker.select.items().reset(towerNames);
    });
  };
  /** Refreshes the current map layer based on the UI widget states. */
  app.refreshMapLayer = function() {
    var imageId = app.picker.select.getValue();
    selectedTower = imageId;
    print('selected tower', selectedTower);
    var targetTower =towers.filter(findTower)[0]
    if(targetTower != null){
      Map.setCenter(targetTower.location[0], targetTower.location[1],zoomLevel)
    }
    i++;
  };
};

/** Creates the app constants. */
app.createConstants = function() {
```

```
  app.COLLECTION_ID = 'LANDSAT/LC08/C01/T1_RT_TOA';
  app.SECTION_STYLE = {margin: '20px 0 0 0'};
  app.HELPER_TEXT_STYLE = {
      margin: '8px 0 -3px 8px',
      fontSize: '12px',
      color: 'gray'
  };

  app.VIS_OPTIONS = {
    'Natural color (B4/B3/B2)': {
      description: 'View Ground features appear in colors similar to their ' +
                   'appearance to the human eye.',
      visParams: {gamma: 1.3, min: 0, max: 0.3, bands: ['B4', 'B3', 'B2']}
    },
    'False color (B7/B6/B4)': {
      description: 'Vegetation is shades of red, urban areas are ' +
                   'cyan blue, and soils are browns.',
      visParams: {gamma: 1.3, min: 0, max: 0.3, bands: ['B7', 'B6', 'B4']}
    },

  };
};
/* creates legends */
var viridis = {min:0, max:25, palette:['cyan', 'green', 'yellow','pink', 'red']};
app.makeLegends = function(viridis){
  var lon = ee.Image.pixelLonLat().select('longitude');
  var gradient = lon.multiply((viridis.max - viridis.min)/100.0).add(viridis.min);
  var legendImage = gradient.visualize(viridis);
  var thumb = ui.Thumbnail({
```

```
    image: legendImage,
    params: {bbox: '0,0,100,8', dimensions:'256x20'},
    style: {position: 'bottom-center'}
    });
    var panel2 = ui.Panel({
      widgets: [
        ui.Label('Low'),
        ui.Label({style: {stretch: 'horizontal'}}),
        ui.Label('High')
        ],
        layout: ui.Panel.Layout.flow('horizontal'),
        style: {stretch: 'horizontal', maxWidth: '270px', padding:'0px 0px 0px 0px'}
      });
      return ui.Panel().add(panel2).add(thumb);
  }
  /** Create a button to unhide the panel.**/
function addPanelButton(){
var button = ui.Button({
  label: 'Open selector panel',
  onClick: function() {
    button.style().set('shown', false);
    app.main.style().set('shown', true);
  }
});
var listenerId = Map.onClick(function() {
      app.main.style().set('shown', false);
      button.style().set('shown', true);
    });
Map.add(button);
```

```
button.style().set('shown', false);
}
/**Create inspector panel**/
app.createInspector=function (){
var inspector = ui.Panel({
  layout: ui.Panel.Layout.flow('vertical')
  });
inspector.add(ui.Label('Click the map to get Location'))
Map.onClick(function(coords){
  inspector.clear();
  inspector.style().set('shown',true);
  inspector.add(ui.Label('Loading...', {color: 'gray'}))

  var loc = coords.lat.toFixed(6)+', '+coords.lon.toFixed(6)+'(latitude, longitude)';
  inspector.clear();
  inspector.add(ui.Label('Clicked Location:', {fontWeight: 'bold',}))
  inspector.add(ui.Label({
    value: loc,
    style: {stretch: 'vertical'}
    }));

  inspector.add(ui.Button({
    label: 'close',
    onClick: function(){inspector.style().set('shown',false)}
    }))
  })
  return inspector;
}
app.boot = function() {
```

```
  app.createConstants();
  app.createHelpers();
  app.createPanels();
  app.main = ui.Panel({
    widgets: [
      ui.Label('Click on the map to get the point location.', {fontSize: '0.9em'}),
      app.intro.panel,
      app.picker.panel,
      app.makeLegends(viridis),
      app.createInspector()
    ],
    style: {width: '280px', padding: '1px'
    }
  });
  Map.setCenter(towers[9].location[0], towers[9].location[1],zoomLevel)
  ui.root.insert(0, app.main);
  app.applyFilters();
};
app.boot();
```